

Off-grid Schutzsensorik

Entwicklung und Implementierung einer LoRaWan-Kommunikation für mobile Sen- sorsysteme in gefährlichen Einsatzgebieten

vorgelegt von

Nguyen Trong Khoa

EDV.Nr.:951 990

dem Fachbereich VII – Elektrotechnik – Mechatronik – Optometrie
der Berliner Hochschule für Technik Berlin

vorgelegte Bachelorarbeit

zur Erlangung des akademischen Grades

Bachelor of Engineering (B.Eng.)

im Studiengang

Humanoide Robotik

Tag der Abgabe 28. Juni 2026

Version 0.1α
letzte Änderung: 28. Juni 2026

Gutachter

Prof. Dr. Peter Gober Berliner Hochschule für Technik

Prof. Dr. Manfred Hild Berliner Hochschule für Technik



Studiere Zukunft

Entwurf

Kurzfassung

Die Kurzfassung gibt ein kurzes und prägnantes Bild der gesamten Arbeit. Sie soll den Leser neugierig machen und klarmachen, was zu erwarten ist. Erreichte Ergebnisse werden kurz umrissen.

Entwurf

Erklärung

Ich versichere, dass ich diese Abschlussarbeit ohne fremde Hilfe selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel benutzt habe. Wörtlich oder dem Sinn nach aus anderen Werken entnommene Stellen sind unter Angabe der Quellen kenntlich gemacht.

Datum

Unterschrift

Entwurf

Entwurf

Inhaltsverzeichnis

1	Einleitung	3
1.1	Mögliche Einsatzszenarien	3
1.1.1	Industrie und Bergbau	3
1.1.2	Katastrophenschutz und Rettungswesen	4
1.1.3	Umweltmonitoring in Extremgebieten	4
1.1.4	Operative Anforderungen an die Schutzsensorik	4
1.2	Grenzen konventioneller Kommunikationsinfrastrukturen	5
1.3	LPWAN als Lösungsansatz für Off-Grid-Einsatz	6
1.3.1	SigFox vs NB-IoT vs LoRaWAN: Ein Vergleich	6
1.3.2	Fazit	6
2	Grundlagen	9
2.1	LoRa-Modulation	9
2.1.1	Allgemeines	9
2.1.2	Wichtige Kenngrößen und Formeln	10
2.1.3	Frequenzbänder und regulatorische Rahmenbedingungen	13
2.2	LoRaWAN – Network Layer	13
2.2.1	Endgeräte	13
2.2.2	Gateways	14
2.2.3	Network-Server	15
2.2.4	Join-Server	16
3	Systementwurf und Implementierung	19
3.1	Hardwareauswahl	19
3.2	Erste Tests	20
3.3	Hardware-Entwicklung	25
3.3.1	Schematic und Komponentenauswahl	25

3.3.2	PCB-Layout	43
3.3.3	Fertigung und Bestückung	49
3.4	Firmware-Entwicklung	50
3.4.1	Wahl des Betriebssystems: Zephyr-RTOS	50
3.4.2	Zephyr-Architektur: Devicetree, KConfig, Treibermodelle	51
3.4.3	LoRaWAN-Netzwerkanbindung	54
3.4.4	Integration von Peripherien	57
3.5	Ergebnisse	58
3.5.1	Validierung der Netzwerkanbindung	58
3.5.2	Kartografische Evaluierung der TTN-Netzwerkabdeckung	59
3.5.3	Energieprofiling	61
3.6	Fazit und Ausblick	61
3.6.1	Ausblick	61
A	Angehängtes: Die Dateien des Pakets	63
	Literatur- und Quellenverzeichnis	65

Abbildungsverzeichnis

3.1	Erster Testaufbau	21
3.2	Einstellung Antennenlänge	21
3.3	LoRa-Rechner	22
3.4	LoRa-Verkehr in SDR#	23
3.5	Blockdiagramm zum Empfangen von LoRa-Testpaketen in GNU-Radio	23
3.6	Dekodierte Nutzdaten in GNU-Radio-Terminal	24
3.7	Blockschaltbild PCB-Prototyp	25
3.8	Ladeblock mit USB-C-Anschluss und BQ24074	27
3.9	Energiegewinnung durch Solarzelle mit SPv1050	29
3.10	Batterie-SoC-Tracking mit MAX17048	30
3.11	MCU-Schaltung mit STM32WL55	31
3.12	Taktversorgung mit HSE	32
3.13	Überprüfung der TCXO-Frequenz über MCO-Ausgang	32
3.14	Taktversorgung mit LSE	33
3.15	TX-Impedanzanpassung	34
3.16	RX-Impedanzanpassung	35
3.17	Entkopplungsschema der MCU	37
3.18	MCU-Reset-Schaltung	38
3.19	BOOT0-Konfiguration	39
3.20	I2C-Schnittstelle	39
3.21	ESD-Schutz für Testheaders	40
3.22	SPI-Flash-Speicher	41
3.23	Status-LED	42
3.24	PCB-Stackup mit vier Lagen: Top (L1), Inner 1 (L2), Inner 2 (L3), Bottom (L4)	44
3.25	Optimale Power-Plane-Auslegung laut AN5407	45
3.26	Impedanzberechnung in KiCad	46

3.27	Layout-Richtlinien für impedanzkontrollierte Leiterbahnen	47
3.28	2D-Ansicht der fertigen Leiterplatte	48
3.29	3D-Ansichten der fertigen PCB	49
3.30	Hierarchische Baumstruktur des beispielhaften Devicetrees.	53
3.31	Registrierung eines neuen Endgeräts im LoRaWAN-Netzwerk bei The Things Network	55
3.32	Screenshot von TTN-Konsole nach erfolgreichem Join-Request	59

Entwurf

Acronyms

ABP Activation by Personalization. 16, 17

ADR Adaptive Data Rate. 16

AppSKey Application Session Key. 16

AS Application Server. 13

BLE Bluetooth Low Energy. 6

BPSK Binary Phase Shift Keying. 20

BW Bandwidth (Bandbreite). 10

CR Coderate. 10, 11

CSS Chirp Spread Spectrum. 9

DevAddr Device Address. 16, 17

DevEUI Device EUI. 16, 17

ED End Device. 13

EUI Extended Unique Identifier. 16

FSK Frequency Shift Keying. 9, 20

ISM Industrial, Scientific, and Medical. 13

JS Join Server. 13, 16

LoRa Long Range. 9

LoRaWAN Long-Range Wide-Area-Network. 9, 13

LPWAN Low Power Wide Area Network. 6

LTE Long Term Evolution. 5

MAPL Maximum Allowable Path Loss. 12

MCU Microcontroller Unit. 13

NS Network Server. 13, 15

NwkSKey Network Session Key. 16

OTAA Over-the-Air Activation. 16

RSSI Received Signal Strength Indicator. 12

RX Receive (Empfangen). 14

SF Spreading Factor. 10

SNR Signal-to-Noise Ratio. 12

SoC System-on-Chip. 20

ToA Time-on-Air. 11

WLAN Wireless Local Area Network. 5

Entwurf

Kapitel 1

Einleitung

Dieses Kapitel widmet sich der Frage, warum LoRaWAN die Technologie der Wahl ist. Zunächst werden repräsentative Einsatzszenarien analysiert, um die spezifischen Anforderungen an Schutzsensorik in Gefahrgebieten zu definieren. Darauf aufbauend wird dargelegt, warum konventionelle Kommunikationstechnologien in diesen Umgebungen strukturell versagen. Abschließend erfolgt ein Vergleich von LPWAN-Technologien, um die Entscheidung zugunsten LoRaWAN für das in dieser Arbeit entwickelte System zu begründen.

1.1 Mögliche Einsatzszenarien

1.1.1 Industrie und Bergbau

Der Untertagebau gehört zu den gefährlichsten Arbeitsbereichen weltweit. Gesteinsschichten verursachen eine extreme Signaldämpfung, die herkömmliche Funksysteme bereits nach wenigen Metern unbrauchbar macht, während gleichzeitig die kontinuierliche Überwachung toxischer und explosiver Gase, insbesondere Methan und Kohlenmonoxid, überlebenswichtig ist. Methan akkumuliert in schlecht belüfteten Stollen und kann bei Erreichen der unteren Explosionsgrenze von 5 % Volumenanteil in Luft zu verheerenden Explosionen führen [1]. Ein Ausfall der Gasüberwachung in explosionsgefährdeten Zonen nach ATEX-Richtlinie 2014/34/EU kann damit katastrophale Folgen haben.

Auch in chemischen Anlagen und Raffinerien ist die zuverlässige Detektion von Schadstofflecken eine sicherheitskritische Aufgabe. Weitläufige Industrieareale mit Hindernissen, Metallstrukturen und variierenden Umgebungsbedingungen erschweren den Aufbau einer zuverlässigen Kommunikationsinfrastruktur zusätzlich.

1.1.2 Katastrophenschutz und Rettungswesen

Einsatzkräfte des Katastrophenschutzes und der Feuerwehr operieren per Definition in Umgebungen, in denen die Infrastruktur entweder nicht vorhanden oder durch das Ereignis selbst zerstört worden ist. Bei Erdbeben werden Basisstationen und Stromversorgung häufig vom selben Ereignis getroffen wie die Einsatzkräfte; bei Waldbränden befinden sich Helfer in weitläufigen, abgelegenen Gebieten ohne Mobilfunkabdeckung [2].

In diesen Szenarien ist ein rasch deployierbares, autarkes Kommunikationsnetz erforderlich, das ohne externe Stromversorgung oder Internetanbindung funktioniert. Neben der reinen Datenübertragung von Vitaldaten und Schadstoffmesswerten ist die Personenverfolgung eine wesentliche Funktion: die Einsatzleitung muss nämlich jederzeit wissen, wo sich einzelne Truppen befinden, um im Gefahrenfall koordiniert reagieren zu können.

1.1.3 Umweltmonitoring in Extremgebieten

Bei der Überwachung von Vulkanen, Gletschern oder Hochwasser-Pegelständen in infrastrukturschwachen Regionen müssen Sensorknoten über Monate oder Jahre hinweg energieautark arbeiten. Physischer Zugang für Wartungsarbeiten ist oft lebensgefährlich oder logistisch unmöglich, denn ein Sensorknoten an einem Vulkanhang etwa kann nicht für einen regulären Batteriewechsel aufgesucht werden.

Dies setzt extrem niedrige mittlere Stromverbräuche voraus, die nur durch konsequente Nutzung von Tiefschlafmodi und ereignis- oder zeitgesteuerte Übertragungsstrategien erreichbar sind. Gleichzeitig darf die Reichweite nicht durch den Energiesparbetrieb so weit eingeschränkt werden, dass die Datenübertragung an die (oft ebenfalls entfernte) Basisstation nicht mehr zuverlässig möglich ist.

1.1.4 Operative Anforderungen an die Schutzsensorik

Aus den drei analysierten Szenarien lassen sich die zentralen technischen Anforderungen ableiten, die ein geeignetes Kommunikationssystem für Off-Grid-Schutzsensorik erfüllen muss:

Die drei übergeordneten Anforderungen, die alle Szenarien gemeinsam haben, sind damit:

- **Infrastrukturunabhängigkeit:** das Netz muss ohne externe Basisstationen oder Internetanbindung vollständig funktionsfähig sein.
 - **Hohe Reichweite bei starker Dämpfung:** das Signal muss Gestein, Beton und Vegetation durchdringen.
-

Tabelle 1.1: Vergleich der Einsatzszenarien hinsichtlich operativer Anforderungen

	Industrie / Bergbau	Katastrophen- schutz	Umwelt- monitoring
Infrastruktur	Keine (Untertagebau)	Zerstört / keine	Keine
Reichweite	100 m bis 500 m	500 m bis 2000 m	2 km bis 15 km
Batterielaufzeit	8–24 h	4–48 h	Monate–Jahre
Latenztoleranz	Niedrig (< 5 s)	Mittel (< 30 s)	Hoch (< 30 min)
Mobilität der Knoten	Hoch	Hoch	Keine

- **Maximale Energieeffizienz:** der Stromverbrauch der Endgeräte muss einen Akkubetrieb über den gesamten Einsatzzeitraum ermöglichen.

1.2 Grenzen konventioneller Kommunikationsinfrastrukturen

Gängige drahtlose Kommunikationstechnologien wie WLAN, Bluetooth und Mobilfunk der vierten und fünften Generation sind für den Einsatz in erschlossenen, infrastrukturell versorgten Umgebungen optimiert. Gegenüber den in Abschnitt 1.1 definierten Anforderungen weisen sie strukturelle Defizite auf, die einen zuverlässigen Einsatz in Gefahrgebieten ausschließen.

- **Reichweite:** WLAN nach IEEE 802.11 erreicht im Innenbereich typischerweise 30 bis 100 m und erfordert die Installation von Access-Points in regelmäßigen Abständen – eine Voraussetzung, die im Untertagebau oder in Katastrophengebieten nicht erfüllbar ist. Bluetooth ist mit einer Reichweite von bis zu 100 m (Klasse 1) für weiträumige Gefahrgebiete ebenfalls ungeeignet.
- **Infrastrukturabhängigkeit:** Mobilfunknetze bieten zwar eine großflächige Abdeckung, weisen jedoch einen systemimmanenten *Single-Point-of-Failure* auf: Basisstationen werden häufig durch dasselbe Ereignis zerstört oder überlastet, das den Notfall verursacht, wie zum Beispiel Erdbeben, Überschwemmungen oder Brände [2]. Private LTE- oder 5G-Campusnetze könnten diese Abhängigkeit prinzipiell aufheben, erfordern jedoch lizenziertes Spektrum, einen dedizierten Core-Network-Betrieb und eine erhebliche Infrastrukturinvestition, die für mobile Einsatzszenarien nicht praktikabel ist.
- **Energieverbrauch:** Die für hohe Datenraten optimierten Protokolle, insbesondere LTE

und 5G, verursachen einen hohen Strombedarf, der den Langzeitbetrieb kleiner, batteriebetriebener Sensorknoten unmöglich macht. Ein LTE-Modul im Sendebetrieb verbraucht typischerweise mehrere hundert Milliampere, was bei einer 1000 mAh-Zelle eine Betriebsdauer von wenigen Stunden ergäbe.

Tabelle 1.2: Eignung gängiger Funktechnologien für Off-Grid-Gefahrgebiete

Technologie	Reichweite	Infrastruktur-frei	Energieeffizienz	Geeignet
WLAN (802.11)	30 m bis 100 m	Nein	Mittel	Nein
BLE (Bluetooth)	bis 100 m	Ja	Hoch	Nein
LTE / 5G	1 km bis 10 km	Nein	Gering	Nein

Aus Tabelle 1.2 ist ersichtlich, dass keine konventionelle Kommunikationstechnologie alle drei genannten Kernanforderungen gleichzeitig erfüllen können. Für den Einsatz in Gefahrgebieten bedarf es also einer Alternative.

1.3 LPWAN als Lösungsansatz für Off-Grid-Einsatz

Low Power Wide Area Network (LPWAN)-Technologien wurden entwickelt, um genau diese Lücke zu schließen. Sie kombinieren Reichweiten von mehreren Kilometern mit einem Energieverbrauch, der einen jahrelangen Akkubetrieb ermöglicht, jedoch auf Kosten einer bewusst niedrig gehaltenen Datenrate, die für die periodische Übertragung von Sensormesswerten jedoch vollkommen ausreichend ist. Im Folgenden werden die drei bekanntesten etablierten LPWAN-Standards, nämlich **SigFox**, **NB-IoT** und **LoRaWAN**, sich gegenübergestellt.

1.3.1 SigFox vs NB-IoT vs LoRaWAN: Ein Vergleich

Chaudhari und Borkar haben sich in [3] intensiv mit den Vor- und Nachteilen diverser LPWAN-Technologien auseinandergesetzt, sowohl aus theoretischer als auch aus praktischer Sicht. Tabelle 1.3 stellt die wichtigsten Erkenntnisse ihrer Analyse dar, mit Fokus nur auf die drei o.g. Standards.

1.3.2 Fazit

Die Vielfältigkeit der LPWAN-Welt zeugt einerseits von den zahlreichen Möglichkeiten dieser Technologie, andererseits erschwert sie die Auswahl einer optimalen Lösung. Unsere hollisti-

Tabelle 1.3: Detaillierter Vergleich führender LPWAN-Standards

Kriterium	Sigfox	NB-IoT	LoRaWAN
<i>Physikalische Schicht</i>			
Frequenzband	433, 868 MHz (EU), 915 MHz (US)	900 MHz	433, 868 MHz (EU), 915 MHz (US)
Spektrum	Unlizenziert	Lizenziert (LTE)	Unlizenziert
Max. Datenrate	0.1 kbps	250 kbps	50 kbps
Reichweite	bis 40 km	bis 15 km	bis 20 km
<i>Link-Schicht</i>			
Max. Paketgröße	12 Bytes (UL) / 8 Bytes (DL)	125 B	51 Bytes (UL) / 14 Bytes (DL)
Latenz	10 s	0.2 s	5 s
Duty-Cycle-Beschränkung	1 % (868 MHz EU)	Keine (lizenziert)	1 % (868 MHz EU)
<i>Netzwerkarchitektur</i>			
Topologie	Stern (öffentlich)	Stern (öffentlich)	Stern oder Mesh (privat möglich)
Betriebsart	Nur öffentlich	Nur öffentlich (Provider)	Öffentlich oder privat
Internetanbindung erforderlich	Ja	Ja	Nein
<i>Entwicklung, Inbetriebnahme & Kosten</i>			
Kommunikationsmodul	ca. 10€	ca. 70€	ca. 10€
Gateway-Kosten	n/a (öffentl. Netz)	n/a (öffentl. Netz)	ca. 200€
Datenplan (min./max.)	1€ / 14€	6€ / 50€	5€ / 20€
Entwicklungs-Kits	Ab 30€	Ab 45€	Ab 30€
Dokumentation	Vollständig öffentlich	Teilweise öffentlich	Vollständig öffentlich

sche Betrachtung der Anforderungen aus Abschnitt 1.1 und der technischen Eigenschaften der drei führenden LPWAN-Standards zeigt, dass LoRaWAN die beste Passung für die in dieser Arbeit adressierten Off-Grid-Gefahrgebiete bietet. Im nächsten Kapitel werden die theoretischen Grundsteine von LoRa und LoRaWAN gelegt, um die spätere Implementierung und Evaluierung des Prototyps zu fundieren.

Entwurf

Kapitel 2

Grundlagen

Dieses Kapitel legt das theoretische Fundament für die in den folgenden Kapiteln beschriebene Entwicklung. Es gliedert sich in fünf Abschnitte: Zunächst werden die physikalischen Grundlagen der LoRa-Modulationstechnologie erläutert, gefolgt von einer Darstellung des LoRaWAN-Protokollstacks und seiner Netzwerkarchitektur. Anschließend werden Sicherheitsmechanismen, Energieeffizienz sowie Methoden der RF-Planung und Signalausbreitung behandelt. Abschließend wird ein kurzer Überblick über die Normierungsanforderungen im Bereich Schutzsensorik gegeben.

2.1 LoRa-Modulation

2.1.1 Allgemeines

LoRa ist eine von der Firma Semtech entwickelte Modulationstechnologie und bildet die physikalische Grundlage des LoRaWAN-Protokolls. Es basiert auf der Chirp Spread Spectrum (CSS)-Modulation, bei der das zu übertragende Signal auf ein breiteres Frequenzband gespreizt wird, indem sogenannte Chirps-Signale mit kontinuierlich steigender oder fallender Frequenz als Trägersymbole verwendet.

Ursprünglich für Militäranwendungen gedacht, hat das CSS-Verfahren in den letzten 20 Jahren in der Nachrichtentechnik zunehmend an Bedeutung gewonnen dank seiner hervorragenden Eigenschaften¹:

Niedriger Energieverbrauch Ähnlich wie FSK besitzt LoRa eine konstante Einhüllende, wodurch kostengünstige, energieeffiziente Leistungsverstärker verwendet werden können.

¹ „LoRa™ Modulation Basics,“ Semtech, Application Note AN1200.02.

Hohe Störfestigkeit Da ein LoRa-Symbol lang und ausgespreizt ist, haben kurz Rauschimpulse nur einen verschwindenden Einfluss auf die Demodulation. Aus diesem Grund sind LoRa-Signale sehr resistent gegenüber In-Band- und Out-of-Band-Interferenzen. Eine Gleichkanalsunterdrückung von über 20 dB kann erreicht werden, verglichen mit nur -6 dB bei FSK.

Mehrwegeausbreitung und Fading Das breitbandige Chirp-Signal bietet eine hohe Immunität gegenüber Mehrwegeausbreitung und Fading, was den Einsatz in urbanen und suburbanen Umgebungen begünstigt.

Hohe Reichweite Für eine gegebene Sendeleistung kann bei LoRa die Reichweite bis zu 4 mal so groß im Vergleich zu FSK.

2.1.2 Wichtige Kenngrößen und Formeln

Modulationskenngrößen

Um die Übertragungsqualität eines LoRa-Links zu charakterisieren und zu optimieren, sind grundlegende Parameter maßgeblich, unter anderem der Spreading Factor (SF), die Bandbreite (BW) sowie die Coderate (CR). Ihre Wechselwirkung bestimmt den Trade-off zwischen Reichweite, Datenrate und Energieverbrauch, also die drei zentralen Designziele des in dieser Arbeit entwickelten Systems.

Der SF gibt die Anzahl der Bits pro Symbol bzw Chirp an und kann Werte von 7 bis 12 annehmen. Wenn zum Beispiel mit SF12 übertragen wird, enthält jeder Chirp 12 Bits. Es gibt in diesem Fall also $2^{12} = 4096$ mögliche Symbole, die jeweils eine eindeutige Bitfolge repräsentieren. Die Übertragungszeit pro Symbol lässt sich berechnen durch:

$$T_{\text{sym}} = \frac{2^{\text{SF}}}{\text{BW}} \quad (2.1)$$

Die Bandbreite (BW) kann 125, 250 oder 500 kHz betragen. Eine größere Bandbreite ermöglicht höhere Datenraten und somit auch schnellere Übertragungen, reduziert jedoch die Empfindlichkeit des Empfängers sowie die Reichweite.

Die Symbolrate F_{sym} kann durch Kehrwertbildung ermittelt werden:

$$F_{\text{sym}} = \frac{1}{T_{\text{sym}}} = \frac{\text{BW}}{2^{\text{SF}}} \quad (2.2)$$

Da jedes Symbol per Definition SF Bits mit sich trägt, ergibt sich die Bitrate R_b zu:

$$R_b = F_{\text{sym}} \cdot \text{SF} = \frac{\text{BW} \cdot \text{SF}}{2^{\text{SF}}} \quad (2.3)$$

Die Time-on-Air (ToA) bzw. Gesamtübertragungszeit eines LoRa-Pakets folgt unmittelbar:

$$T = n_{sym} \cdot T_{sym} = n_{sym} \cdot \frac{2^{SF}}{BW} \quad (2.4)$$

Zwecks Fehlerentdeckung und -korrektur werden zusätzlich zum Payload Redundanzbits übertragen, deren Anteil durch die Coderate (CR) definiert ist. Bei LoRa können CR-Werte von 4/5 bis 4/8 eingestellt werden. Eine CR von 4/6 bedeutet bswp., dass von insgesamt 8 übertragenen Bits nur 4 Nutzdatenbits sind, während die restlichen 2 Bits als Redundanz dienen.

Tabelle 2.1: Übersicht der LoRa-Coderaten

Einstellung	Redundanz-Overhead	Schutzlevel & Anwendung
CR 4/5	25 %	Standard in LoRaWAN; geringster Overhead, ideal für gute Funkbedingungen
CR 4/6	50 %	Erhöhter Schutz bei moderaten Störungen
CR 4/7	75 %	Hohe Robustheit gegen Burstrauschen; deutlich längere Sendezeit
CR 4/8	100 %	Maximaler Schutz

Unter Berücksichtigung der Coderaten lässt sich die effektive Bitrate R_{eff} folgendermaßen ausdrücken:

$$R_{\text{eff}} = R_b \cdot CR = \frac{BW \cdot SF}{2^{SF}} \cdot CR \quad (2.5)$$

Tabelle 2.2 stellt den Zusammenhang zwischen wichtigen Kenngrößen dar. Hierbei wird von einer 125 kHz-BW und einer CR von 4/5 ausgegangen, da dies die Standardkonfiguration für LoRaWAN ist. Die Payloadgröße wird willkürlich auf 50 Bytes festgelegt, weil es dem maximal zulässigen Wert für SF12 entspricht.

Linkqualität und Reichweite

Die bisher vorgestellten Parameter SF, BW und CR beschreiben das LoRa-Signal auf der Senderseite. Um jedoch beurteilen zu können, ob eine Übertragungsstrecke unter realen Bedingungen zuverlässig funktioniert, sind empfängerseitige Kenngrößen sowie eine Abschätzung der maximal überbrückbaren Distanz erforderlich.

Tabelle 2.2: Wichtige LoRa-Parameter in einem Blick

SF	Bitrate [bps]	Empfindlichkeit [dBm]	Time on Air (50 Bytes) [ms]	Typischer Einsatz
SF7	5469	−123	73,14	Kurze Distanz, energiesparend
SF8	3125	−126	128,00	Ausgewogener Betrieb
SF9	1758	−129	227,53	Mittlere Distanz
SF10	977	−132	409,42	Grenzbereich Stadtgebiet
SF11	537	−134	744,88	Ländliche Umgebung
SF12	293	−136	1365,19	Maximale Reichweite

Die **Empfangsempfindlichkeit** gibt die minimale Signalleistung an, bei der ein Empfänger ein Signal noch korrekt demodulieren kann. Sie ist direkt vom gewählten SF abhängig: Ein höherer SF ermöglicht eine bessere Empfangsempfindlichkeit, wie bereits aus Tabelle 2.2 ersichtlich ist.

Der **Received Signal Strength Indicator (RSSI)** ist ein Maß für die empfangene Signalleistung in dBm und wird vom Empfänger nach jeder Übertragung gemessen. Er gibt Auskunft darüber, wie stark das Signal am Empfänger ankommt, berücksichtigt jedoch nicht den Rauschpegel. Ergänzend dazu beschreibt das **Signal-to-Noise Ratio (SNR)** das Verhältnis zwischen Nutz- und Rauschsignal. Während RSSI allein keine zuverlässige Aussage über die Demodulierbarkeit eines Signals erlaubt, ermöglicht erst die gemeinsame Betrachtung beider Größen eine fundierte Bewertung der Linkqualität, insbesondere bei schwachen Signalen nahe der Empfangsempfindlichkeitsgrenze.

Das **Maximum Allowable Path Loss (MAPL)**, oft auch als **Link-Budget** bekannt, beschreibt den maximal tolerierbaren Pfadverlust entlang der Übertragungsstrecke und ergibt sich aus der Differenz zwischen Sendeleistung und Empfangsempfindlichkeit unter Berücksichtigung aller Gewinne und Verluste:

$$\text{MAPL} = P_{\text{TX}} + G_{\text{TX}} + G_{\text{RX}} - L_{\text{Kabel}} - \text{NF} - \text{SNR}_{\text{min}} - \text{Sicherheitsmarge} \quad (2.6)$$

wobei P_{TX} die Sendeleistung in dBm, G_{TX} und G_{RX} die Antennengewinne in dBi, NF das Rauschen des Empfängers und SNR_{min} das minimal tolerierte Signal-zu-Rausch-Verhältnis bezeichnen. Als maximaler Antennengewinn einer einfachen Dipolantenne gilt $G = 2.15$ dBi. Eine Sicherheitsmarge von 5 bis 10 dB wird empfohlen, um Schwundeffekte und nicht modellierte Hindernisse zu kompensieren [4].

Aus dem MAPL lässt sich schließlich unter Verwendung des Freiraum-Pfadverlustmodells (vgl.

Abschnitt ??) die **theoretische Reichweite** der Übertragungsstrecke abschätzen. Für den Chip SX1272 von Semtech zum Beispiel kommt man auf einen theoretischen Wert von 1946 km.

2.1.3 Frequenzbänder und regulatorische Rahmenbedingungen

LoRa nutzt sogenannte ISM-Bänder (für Industrial, Scientific, and Medical), für die keine Lizenz erworben werden muss. In Europa werden die Frequenzbänder 433 MHz und 868 MHz verwendet, in Nordamerika 915 MHz. Da die Frequenzzuweisungen regional verschieden sind, können Geräte, die für den europäischen Markt entwickelt wurden, nicht ohne Anpassungen in anderen Regionen betrieben werden.

Für das 868 MHz-Band in Europa gilt eine Duty-Cycle-Begrenzung: Endgeräte dürfen je nach Subband 0,1 %, 1 % der Zeit senden; für Gateways gilt die Schwelle 10%. In der Praxis bedeutet eine 1 %-Beschränkung bswp., dass ein Gerät nach einer Sendedauer von 1 s mindestens 99 s warten muss, bevor es erneut auf demselben Kanal sendet. Für zeitkritische Warn- und Alarmsysteme ist diese Einschränkung bei der Systemauslegung zwingend zu berücksichtigen.

Die maximale Paketgröße in Europa beträgt 222 Bytes bei hoher Datenrate und 51 Bytes bei niedriger Datenrate. Dabei addiert LoRaWAN stets einen 13-Byte-Overhead zum Nutzdaten-Payload.

2.2 LoRaWAN – Network Layer

Der Begriff Long-Range Wide-Area-Network (LoRaWAN) definiert den Netzwerkstandard, dem die LoRa-Modulationstechnologie zugrunde liegt. Der LoRaWAN-Standard wird von der sogenannten LoRa-Alliance - einem im Jahre 2015 gegründeten Bündnis internationaler Unternehmen - gepflegt und in ihren umfangreichen technischen Spezifikationen dokumentiert. Diese beginnen alle mit dem Präfix "TS", gefolgt von einer dreistelligen Dokumentennummer.

Netzwerkarchitektur

Eine LoRaWAN-Kommunikationskette setzt sich aus drei Gliedern zusammen: Endgeräten/End Device (ED), Gateways und Servern. Die Server-Seite gliedert sich weiter in Network Server (NS), Join Server (JS) und Application Server (AS).

2.2.1 Endgeräte

Auf der untersten Ebene befinden sich die **Endgeräte**, deren Zentraleinheit Mikrokontroller (MCU) mit entsprechender Stromversorgung darstellen. Auf dem MCU ist ein LoRaWAN-Stack

integriert, der als Schnittstelle zur LoRa-Funkeinheit fungiert. Diese übernimmt wie bereits beschrieben die physikalische Modulation und Demodulation der Signale, welche mithilfe einer Antenne in die Luft gesendet bzw. von der Luft empfangen werden können. Zusätzlich können Endgeräte mit anwendungsspezifischen Peripheriegeräten wie Sensoren oder Aktoren ausgestattet sein.

Geräteklassen

Das Dokument *LoRaWAN® L2 1.0.4 Specification (TS001-1.0.4)* definiert drei Geräteklassen (A, B und C), die festlegen, wann ein Endgerät Downlink-Nachrichten, also vom Server zu Endgerät, empfangen kann. Die Klasse kann während des Betriebs gewechselt werden.

Klasse A ist die energieeffizienteste und am weitesten verbreitete Klasse, denn jedes LoRaWAN-Gerät muss sie implementieren. Nach jeder Uplink-Übertragung öffnet das Gerät zwei kurze Fenster (RX1 und RX2) für das Empfangen von Downlink-Nachrichten. Außerhalb dieser Fenster befindet sich das Gerät im Schlafmodus, um den Stromverbrauch zu minimieren. Eine erneute Uplink-Kommunikation darf erst nach Empfang einer Downlink-Nachricht oder nach Ablauf von RX2 erfolgen. Diese Klasse eignet sich besonders für batteriebetriebene Sensoren, die nur gelegentlich Daten senden müssen, wie es bei vielen Schutzsensoren der Fall ist.

Im Vergleich zur Klasse A ist die **Klasse B** flexibler, da sie einen periodischen Empfang durch zeitgesteuerte RX-Fenster ermöglicht, die mittels Gateway-Beacons synchronisiert werden. Dadurch würde unmittelbar der Network Server erfahren, sobald Endgeräte empfangsbereits sind. Dies erhöht die Reaktionsfähigkeit auf Kosten eines höheren Energieverbrauchs. Wenn ein Gerät sich als Klasse-B-Gerät Network-Server gegenüber identifizieren will, muss der entsprechende ClassB-Bit in einem Uplink-Signal gesetzt werden.

Klasse C ermöglicht einen kontinuierlichen Empfang, indem das Endgerät nur während des Sendens nicht empfängt. Das hat konsequenterweise den Nachteil, dass Klasse-C-Geräte für batteriebetriebene Geräte ungeeignet sind und für das Thema Off-Grid-Schutzsensorik eine eher untergeordnete Rolle spielen.

2.2.2 Gateways

Nach den Endgeräten schließen befinden sich direkt im Anschluss die **Gateways**, welche ebenfalls auf der Physical layer wirken. Bei der uplinkmäßigen Übertragung, also von Endgerät zu Server, empfangen sie die Signale der Endgeräte und leiten diese an die Netzwerkservers weiter. Umgekehrt gilt für die Downlink-Richtung, dass Gateways die Signale der Server Endgeräten zur Verfügung stellen. Da Gateways permanent auf allen Kanälen lauschen, können Endgeräte

Tabelle 2.3: LoRaWAN-Klassen und ihrer Merkmale

Klasse	Merkhilfe	Hauptmerkmal	Energieprofil
A	All	Grundfunktion: Muss von allen Geräten unterstützt werden.	Maximal (Schlafmodus standardmäßig)
B	Beacon	Synchronisation durch Gateway-Beacons (erfordert ClassB-Bit).	Medium (periodisches Aufwachen)
C	Continuous	Continuierlicher Empfang (fast immer aktiv).	Minimal (nicht für Batteriebetrieb)

jederzeit senden, solange regulatorische Gegebenheiten es erlauben.

Die Kommunikation zwischen Endgeräten und Gateways basiert auf dem ALOHA-Protokoll. Das bedeutet, dass ein Endgerät nicht an ein bestimmtes Gateway gebunden ist, sondern eine beliebige Anzahl an Gateways in Reichweite nutzen kann, um mit dem Netzwerk Daten auszutauschen. Aufgrund dessen, dass Gateways aus Sicht des LoRaWAN-Protokolls transparent sind, werden sie in technischen Spezifikationen der LoRa-Alliance nur flüchtig erwähnt und größtenteils "wegabstrahiert". Um diese theoretische Transparenz jedoch zu gewährleisten, müsste man bei der Platzierung Gedanken machen. Indoor-Gateways zum Beispiel, also in Innenräumen installierte Gateways, haben eine beschränkte Reichweite, die sie dem hohen Dämpfungsvermögen von Baumaterialien zu verdanken haben. Daher empfiehlt es sich, sie in der Nähe von Fensterfronten zu platzieren, um den Signaldurchgang zu verbessern.

Für den Außeneinsatz sind hingegen Outdoor-Gateways erforderlich, die aufgrund wasserdichter Gehäuse und Hohlraumfilter teurer sind als ihre Indoor-Pendants. In diesem Fall gilt es zu beachten, dass Gateways möglichst hoch montiert werden sollen, um die Sichtlinie zu den Endgeräten zu garantieren und damit die Reichweite zu erhöhen.

2.2.3 Network-Server

Mit den Gateways hört auch der Geltungsbereich von LoRa auf und wir verlassen somit die Bitübertragungsschicht des OSI-Modells. Die Informationspakete gelangen als Nächstes über eine Backhaul-Verbindung (bswp. Ethernet oder Mobilfunk) zu den Servern, von welchen der **Network-Server** (NS) der zentrale Knotenpunkt der LoRaWAN-Sterntopologie ist. An dieser Stelle tritt das Dokument *LoRaWAN® Backend Interfaces Technical Documentation (TS002-1.1.0)* zum Vorschein.

Zu den Aufgaben eines Network-Servers gehören: Überprüfung von Endgerätadressen, Paketauthentifizierung, Anpassung der Datenraten, Weiterleiten von Payloads bzw. Join-Anfragen an entsprechende Application-Servers und Join-Server jeweils. Zusätzlich kümmern sich Network-Server um die Konsolidierung von Nachrichten, die von mehreren Gateways empfangen wurden, sowie um die Überwachung der Netzwerk- und Gateway-Leistung.

Adaptive Data Rate

Adaptive Data Rate (ADR) ist ein Mechanismus des Network Servers zur automatischen Anpassung von Datenrate und Sendeleistung eines Endgeräts. Endgeräte in der Nähe eines Gateways werden auf höhere Datenraten (niedrigere SF-Werte) umgeschaltet, was die Luftzeit verkürzt und die Netzwerkkapazität erhöht. Geräte in größerer Entfernung werden mit niedrigeren Datenraten betrieben, um ausreichende Verbindungsqualität sicherzustellen. Wird ein weiteres Gateway zum Netzwerk hinzugefügt, aktualisieren die Endgeräte ihren SF automatisch. ADR ist ausschließlich für stationäre Geräte geeignet, für mobile Anwendungen wie das hier betrachtete Schutzsensorysystem muss ADR deaktiviert oder durch ein geeignetes mobilitätsbewusstes Verfahren ersetzt werden.

2.2.4 Join-Server

Der **Join-Server** verwaltet den Netzwerkbeitritt bzw. Aktivierung von Endgeräten. Jeder JS verfügt zur Identifikation über eine eindeutige JoinEUI (Extended Unique Identifier), welche von den Endgeräten bei Join-Anfragen verwendet wird. Erfolgreiche Beitrittsanfragen bewirken die Erzeugung von Sitzungsschlüsseln (NwkSKey und AppSKey), somit ist die Anbindung von Endgeräten an das LoRaWAN-Netzwerk abgeschlossen und sie können mit der Datenübertragung beginnen.

Aktivierungsverfahren: OTAA und ABP

Es wird zwischen zwei Aktivierungsverfahren unterschieden: Over-the-Air Activation (OTAA) und Activation by Personalization (ABP).

Bei OTAA initiiert das Gerät beim ersten Start eine *Join Request*, die sowohl seine DevEUI als auch die Join-EUI des Join-Servers enthält. Zusätzlich wird noch eine DevNonce übermittelt, die bei jeder Anfrage inkrementiert werden soll, andernfalls würde der die Aktivierung verweigert. Bei erfolgreicher Aktivierung sendet der Join-Server eine *Join Accept* und stellt dabei die notwendigen Sitzungsschlüssel, Network Session Key (NwkSKey) und Application Session Key (AppSKey), bereit. Das Gerät bekommt außerdem eine eindeutige Device Address (DevAddr) vom Network-

Server zugewiesen. Die LoRa-Alliance empfiehlt OTAA als bevorzugtes Aktivierungsverfahren, da die dynamische Ableitung von Sitzungsschlüsseln für Sicherheit sorgt.

Im Gegensatz zu OTAA werden bei ABP alle benötigten Parameter (DevEUI, DevAddr und Sitzungsschlüssel) vorab im Gerät fest einprogrammiert. Das Gerät kann sofort ohne Join-Prozedur senden. Dieses Verfahren ist einfacher zu implementieren, allerdings dafür weniger flexibel und gilt sicherheitstechnisch als suboptimal. Wenn beispielsweise ein Gerät kompromittiert wird, können die fest eingebauten Schlüssel nicht mehr geändert werden, was zu einem dauerhaften Sicherheitsrisiko führt.

Entwurf

Entwurf

Kapitel 3

Systementwurf und Implementierung

TODO: rewrite when done

Dieses Kapitel beschreibt den Entwurf und die prototypische Umsetzung des Off-Grid-Schutzsensorsystems. Ausgehend von den in Kapitel 1 abgeleiteten Anforderungen werden zunächst die gewählten Hardware-Komponenten vorgestellt und begründet. Anschließend wird die Systemarchitektur auf Software-Ebene beschrieben, bevor auf die konkrete Implementierung der Firmware, des Network-Servers und der Datenvisualisierung eingegangen wird.

TODO: Blockschaltbild des Gesamtsystems einfügen (Sensoreinheit → Gateway → RPi Network-Server)

3.1 Hardwareauswahl

Mikrokontroller

Mikrokontroller Für die Sensoreinheit wird der **STM32WL55CC** von STMicroelectronics eingesetzt, der auf dem Entwicklungsboard **NUCLEO-WL55JC1** verfügbar ist. Der STM32WL55 ist der erste am Markt verfügbare System-on-Chip (SoC), der einen ARM Cortex-M4-Applikationsprozessor, einen ARM Cortex-M0+ für den LoRa-Stack sowie einen integrierten Sub-GHz-Transceiver (SX126x-kompatibel) auf einer einzigen Siliziumfläche vereint. Dadurch werden der Platzbedarf, Kostenpunkt und potenzielle Fehlerquellen an der Schnittstelle zwischen MCU und Funkmodul verringert.

Wichtige Eigenschaften des STM32WL55 für dieses Projekt lassen sich dem Datenblatt entnehmen:

- **Integrierter Sub-GHz-Transceiver:** Unterstützt LoRa (CSS), FSK und BPSK im Frequenzbereich 150–960 MHz, damit nativ kompatibel mit dem europäischen 868 MHz-Band.
-

- **Sendeleistung:** bis zu 22 dBm bei $V_{DDPA} = 3.3\text{ V}$, was in Kombination mit SF12 Reichweiten von mehreren Kilometern im Freifeld ermöglicht. Es ist jedoch Vorsicht geboten, dass in Europa die Sendeleistung auf 14 dBm begrenzt ist.
- **Energieprofil:** Im Deep-Stop-Modus beträgt die Stromaufnahme lediglich $1.08\text{ }\mu\text{A}$, was einen mehrschichtigen Betrieb auf Primärzellen ermöglicht.
- **Zephyr-Support:** Das Board `nucleo_wl55jc` ist offiziell von Zephyr-RTOS unterstützt und begünstigt demgemäß die Firmware-Entwicklung mit dem integrierten LoRaWAN-Stack.
- **Integrierter ST-LINK V3E:** Das NUCLEO-Board enthält einen integrierten Debugger und Programmer, sodass kein separates Programmiergerät erforderlich ist.

Gateway TODO: write sth about DIY RPi as Gateway

Messinstrument Der **Nordic Power Profiler Kit II (PPK2)** ermöglicht eine hochauflösende Strommessung im Bereich von 100 nA bis 1 A und dient damit der Verifikation der theoretischen Energiebudgetberechnungen. Er wird zwischen Spannungsquelle und NUCLEO-Board geschaltet und zeichnet das Stromprofil über die Zeit auf, sodass Schlafphasen, Aufwachvorgänge und Sendepulse direkt sichtbar werden.

Radioempfänger Der **RTL-SDR V3** ist ein kostengünstiger Software Defined Radio (SDR)-Empfänger, der in Kombination mit SDR# (Windows) oder GNU-Radio (Linux) zur Spektrumanalyse eingesetzt wird. Mit dem GNU-Radio-Plugin `gr-lora` können LoRaWAN-Pakete vollständig demoduliert und dekodiert werden, ohne dass ein zweites LoRa-Endgerät oder gar Gateway als Empfänger erforderlich ist.

3.2 Erste Tests

Um die gekauften Komponenten zu überprüfen, wurde auf die Schnelle eine Reihe von Funktionstests durchgeführt. Zuerst erfolgte die Inbetriebnahme des Nucleo-Boards mit Zephyr-RTOS, wobei der bereits gegebene Blinky-Code erfolgreich kompiliert und geflasht werden konnte. Anschließend wurde das Senden eines einfachen Pakets mit dem Nucleo auf Basis des Beispielprogramms `LoRa_send`¹ getestet. In unmittelbarer Nähe des STM32-Nucleos befindet sich der RTL-SDR-Dongle, welcher am einen Ende über USB-Anschluss mit einem Rechner verbunden und

¹<https://github.com/zephyrproject-rtos/zephyr/blob/main/samples/drivers/lora/send/src/main.c>

am anderen Ende mit einer Dipolantenne ausgestattet ist (siehe Abbildung 3.1). Die Stablänge ℓ muss auf 868 MHz abgestimmt sein: $\ell = \frac{\lambda}{4} = \frac{c}{4f} = \frac{3 \times 10^8 \text{ m s}^{-1}}{4 \cdot 868 \times 10^6 \text{ Hz}} \approx 8.64 \text{ cm}$



Abbildung 3.1: Erster Testaufbau



Abbildung 3.2: Einstellung Antennenlänge

Bevor dieser Test beginnen konnte, ist es wichtig, alle Übertragungsparameter im Quellcode sicherheitshalber zu überprüfen. Die Sendeleistung wird auf 4 dBm eingestellt, somit werden in dieser Hinsicht die EU-Grenzwerte eingehalten. Auch der RTL-SDR-Empfänger, welcher durch Tx-Leistungen höher als 10 dBm beschädigt werden könnte, ist damit geschützt. Die ToA sowie der DC erzählen jedoch eine andere Geschichte: wenn man nämlich die restlichen Parameter in einen LoRa-Rechner eingibt (Abbildung 3.3), beträgt die ToA 330 ms. Da in diesem Fall jede Sekunde ein Paket gesendet wird, ergibt sich ein DC von 33%, weit über dem zulässigen Wert von 1% für das EU868-Band. Aus diesem Grund musste im Vorfeld das Übertragungsintervall auf 34 s verlängert werden.

RF	Modem	Packet
Tx Power: 4 dBm	Modulation: LoRa	Preamble Length: 8 Symbols
Receiver Mode: High Sensitivity	Spreading Factor: 10	Header: Enabled
PA Boost: Off	Bandwidth: 125 kHz	Payload Length: 12 Bytes
Ramp Time: 3400 us	Coding Rate: 4/5	CRC: ON
Frequency: 865100000 Hz	Low Data Rate Optimizer: ON	

Abbildung 3.3: LoRa-Rechner

Abbildung 3.4 zeigt das Frequenzspektrum um 865.1 MHz beim erfolgreichen Empfangen in SDR#. Es lässt sich erkennen, dass die meiste Energie im Bereich $865.1 \text{ MHz} \pm \frac{\text{BW}}{2}$ ist. Aufgrund dessen, dass SDR# LoRa-Pakete nicht dekodieren kann, wird ein weiterer Test in GNU-Radio benötigt.

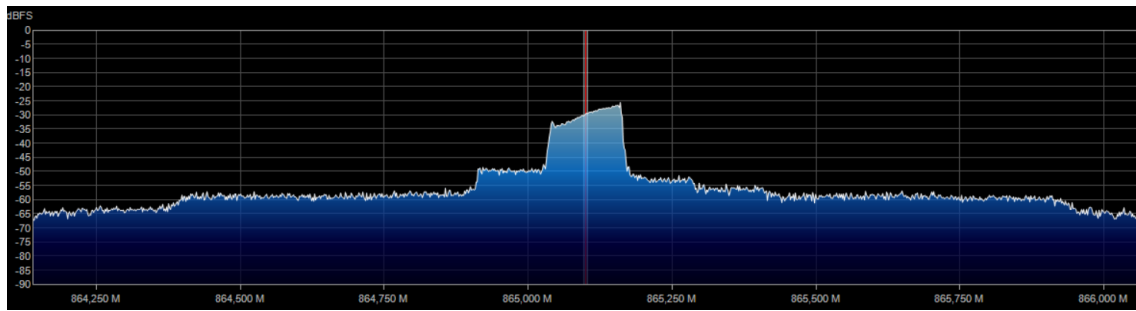


Abbildung 3.4: LoRa-Verkehr in SDR#

GNU-Radio ist eine Open-Source-Software für die Entwicklung von SDR-Anwendungen. Die GUI ähnelt der von Simulinks und basiert auf einem Blockdiagramm, in dem die Blöcke die Signalverarbeitungsschritte darstellen. Ein primitives Flussdiagramm wie in Abbildung 3.5 ist schon ausreichend. Es beginnt mit einem RTL-SDR Source Block, der die Rohdaten vom Empfänger liest und verstärkt. Anschließend werden die Daten durch einen LoRa-Rx-Block geführt, welcher die LoRa-Pakete demoduliert und dekodiert. Die LoRa-Blöcke kommen nicht vorinstalliert mit GNU-Radio und müssen über das Erweiterungsmodul `gr-lora` (entwickelt von Tapparel, Afsiadis, Mayoraz u. a.) eingebunden werden. Die dekodierten Payloads werden im Terminal ausgegeben (Abbildung 3.6)

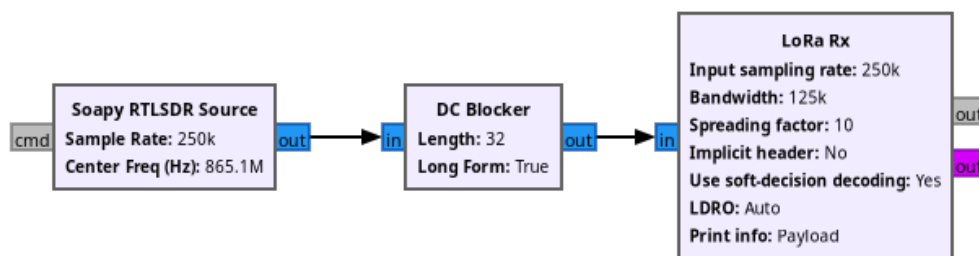
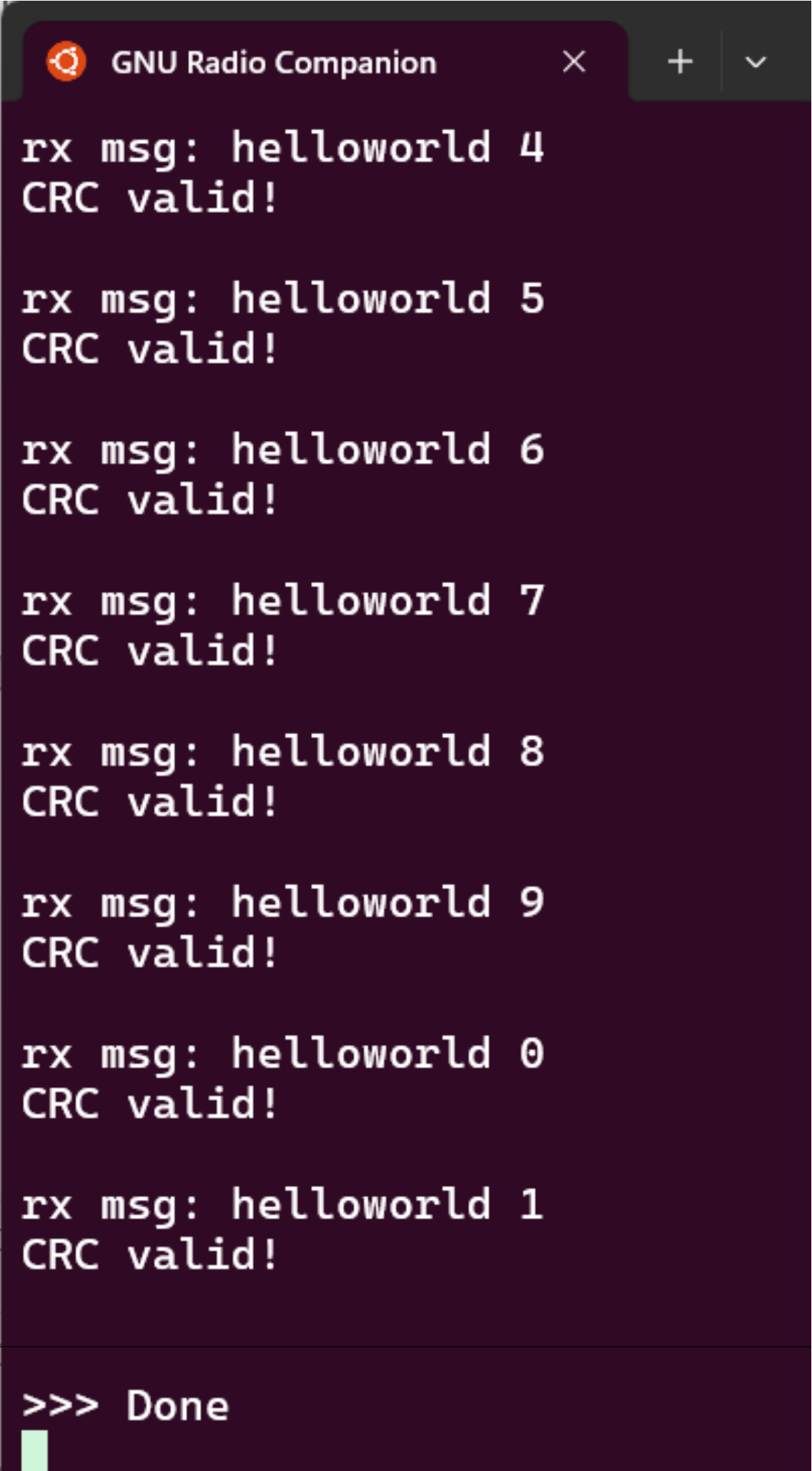


Abbildung 3.5: Blockdiagramm zum Empfangen von LoRa-Testpaketen in GNU-Radio

A screenshot of a terminal window titled "GNU Radio Companion". The window has a dark purple background and a light green cursor at the bottom. The terminal displays a series of received messages, each consisting of a line "rx msg: helloworld" followed by a number and a line "CRC valid!". The numbers are 4, 5, 6, 7, 8, 9, 0, and 1, in that order from top to bottom. The window has a standard macOS-style title bar with a close button (X), a plus button (+), and a dropdown arrow (v).

```
GNU Radio Companion  
rx msg: helloworld 4  
CRC valid!  
  
rx msg: helloworld 5  
CRC valid!  
  
rx msg: helloworld 6  
CRC valid!  
  
rx msg: helloworld 7  
CRC valid!  
  
rx msg: helloworld 8  
CRC valid!  
  
rx msg: helloworld 9  
CRC valid!  
  
rx msg: helloworld 0  
CRC valid!  
  
rx msg: helloworld 1  
CRC valid!  
  
>>> Done
```

3.3 Hardware-Entwicklung

3.3.1 Schematic und Komponentenauswahl

Blockschaltbild

Im nächsten Schritt soll ein maßgeschneiderter PCB-Prototyp entworfen und gefertigt werden. Abbildung 3.7 stellt die Schaltung blockschaltbildmäßig dar. Im Zentrum befindet sich der STM32WL55 mit seinem integrierten LoRa-Transceiver, der über einen SMA-Stecker mit der externen 868MHz-Antenne angeschlossen ist. Mithilfe eines RF-Schalters wird zwischen Empfangs- und Sendemodus gewechselt. Die Steuersignale für den RF-Schalter (RF_SW_V1 und V2 im Blockschaltbild) kommen von der MCU und werden über Pullup- sowie Pulldown-Widerstände so konfiguriert, dass per Default das Empfangen aktiviert ist, da fahrlässiges Senden ohne Antenne das Funkmodul beschädigen könnte.

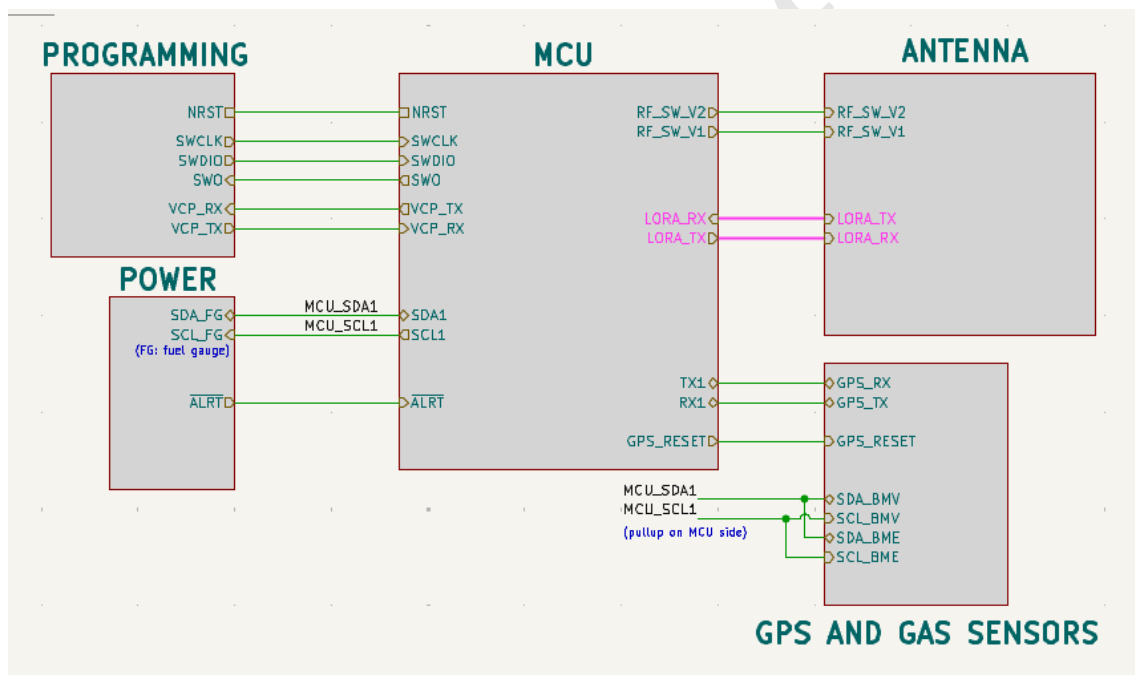


Abbildung 3.7: Blockschaltbild PCB-Prototyp

Als Programmierschnittstelle wird ein STDC14-Header vorgesehen, über den sich das Board mit einem STLinkV3-Minie verbinden kann. Der Einfachheit halber wurde für das Protokoll Serial Wire Debug (SWD) entschieden, denn dieses nimmt lediglich zwei Pins in Anspruch (SWDIO und SWCLK) im Vergleich zu JTAG, welches zusätzlich noch TMS, TDI und TDO benötigt. In diesem Design wird darüber hinaus von den Pins SWO (Serial Wire Output) sowie VCP-TX und RX (Virtual COM Port) des STLinks Gebrauch gemacht, um die Firmware-Entwicklung zu erleichtern.

Im Bereich Sensoren finden sich auf der PCB insgesamt drei. Zwei Gassensoren, nämlich BME680 und BMV080 von Bosch dienen der Luftmessung und kommunizieren mit dem STM32 über einen geteilten I2C-Bus. Der BME680 ermöglicht das Messen von Temperatur, Luftfeuchtigkeit, Luftdruck und Gaswiderstand erschließt und eignet sich demzufolge für allgemeine IoT-Anwendungen. Hingegen ist der BMV080 auf die Erfassung von gesundheitsschädlichen Feinstaubteilchen spezialisiert und liefert aussagekräftige Informationen über die Konzentration von PM1, PM2.5 und PM10 in der Luft. Das letzte Glied des Sensortrios bildet das GPS-Modul L80-R von Quectel. Es hat die Aufgabe, die Reichweite der Antenne zu bereits installierten Gateways zu bestimmen, indem die eigenen GPS-Koordinaten mit denen der Gateways verglichen werden. Der Datenaustausch zwischen GPS-Modul und MCU erfolgt über UART.

Das Projekt wird von einer LiPo-Batterie mit einer Kapazität von 1500 mAh versorgt, die über einen USB-C-Anschluss aufgeladen werden kann. Für einen autarken Betrieb steht ebenfalls eine kleine Solarzelle zur Verfügung.

Wichtige Teilschaltungen

Die vollständige Schaltung wird hier aus Platzgründen nicht aufgeführt, sondern im Anhang A. Wir werden uns in diesem Abschnitt ausschließlich den Teilschaltungen **POWER** bzw. **MCU** widmen.

Die Versorgungsschaltung gliedert sich in drei Blöcke: Ladeblock, Energy-Harvesting aus einer Solarzelle Tracking des Batterie-SoC (state-of-charge) Das Laden des LiPo-Akkus erfolgt wie bereits erwähnt über einen 6-poligen USB-C-Anschluss, der an seinem Ausgangspin VBUS 5V und maximal 900mA bereitstellt. Der 5V Ausgang dient zugleich als Eingang für den Ladechip BQ24074 von Texas Instruments, welcher u.A den Ladestrom als Funktion der Batteriespannung regelt. Durch den 2K-Widerstand R4 wird der Ladestrom auf ca. 880mA begrenzt, um die Grenze des USB-Steckers zu berücksichtigen.

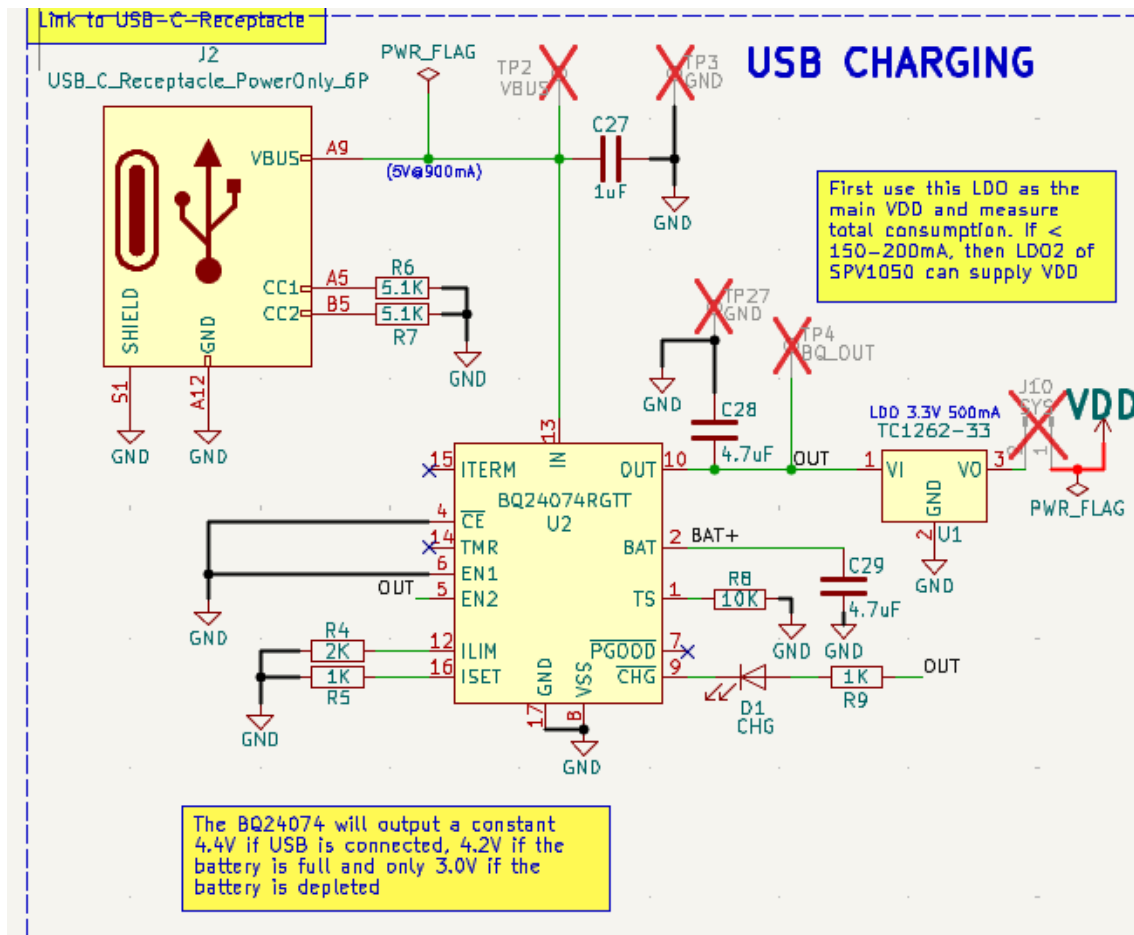


Abbildung 3.8: Ladeblock mit USB-C-Anschluss und BQ24074

Der BQ247074 liefert am Ausgang (Pin OUT) laut Datenblatt 4,2V bei vollem und 3,0V bei leerem Akku. Aus diesem Grund wird an dieser Stelle zusätzlich noch ein LDO eingesetzt, um die Spannung auf 3,3V zu regeln. Der Ausgang VO des LDOs versorgt das gesamte System mit Strom, einschließlich der MCU und sämtlicher Sensoren. LDOs haben gegenüber Schaltreglern den Vorteil, dass sie weniger Rauschen erzeugen, was für die empfindlichen Sensoren sowie die RF-Schaltung von Vorteil ist.

Um die richtige Auswahl des LDOs treffen zu können, muss im Vorfeld der maximale Strombedarf des Systems grob abgeschätzt werden. Die energieaufwändigsten Komponenten sind die Sensoren (samt GPS) sowie das LoRa-Funkmodul auf dem STM32WL55, insbesondere im TX-Modus. Tabellen 23 und 25 im Datenblatt der MCU geben einen maximalen Stromverbrauch von 130mA an, wovon 120mA auf die Sendephase (bei 22 dBm) entfallen. Das L80-R GPS-Modul zieht 25mA im Acquisition-Modus, während der BME680 vernachlässig wenig Strom frisst. Der BMV080 verbraucht allerdings bis zu $\frac{180mW}{3.3V} = 54,5mA$ im kontinuierlichen Messmodus.

Auf Basis dieser Informationen hat sich der TC1262-33 von Microchips als Sieger durchgesetzt.

Tabelle 3.1: Abschätzung des maximalen Strombedarfs (Worst-Case)

Komponente	Modus / Bedingung	Stromverbrauch (max.)
STM32WL55 (Radio)	LoRa TX @ 22 dBm	120,0 mA
STM32WL55 (MCU)	Core + Peripherie	10,0 mA
L80-R GPS Modul	Acquisition	25,0 mA
BMV080	Kontinuierliche Messung	54,5 mA
BME680	Gas-Messung (Peak)	< 2,0 mA
Gesamt	Maximaler Peak	211,5 mA

Er kann maximal 500mA liefern, was einem großzügigen Sicherheitsfaktor von mehr als 2 entspricht. Aufgrund der geringen Dropout-Spannung (275mV@211,5mA) sowie des bescheidenen Stromverbrauchs (80-130µA) ist der TC1262-33 ideal für batteriebetriebene Anwendungen. Zwischen dem LDO und VDD ist zusätzlich noch ein Stecker (J10/SYS in Abbildung 3.8) zu sehen, wodurch der tatsächliche Energieverbrauch des Boards bei Bedarf mit dem PPK2 nachgemessen werden kann.

Die USB-Ladefunktion ist zwar praktisch für den ersten Prototyp zum schnellen Aufladen, aber in Gefahrgelieten oder schwer zugänglichen Umgebungen könnte sie sich als Schwachstelle erweisen. Ein zuverlässiger Mechanismus zur Energieernte im autarken Betrieb erweist sich also als notwendig. Das gelingt über eine kleine Solarzelle, über die der LiPo-Akku aufgeladen werden kann. Das Laden und Regeln übernimmt in diesem Fall nämlich der Chip SPV1050 von ST Microelectronics. Der Schaltung in Abbildung ?? liegt die Application Note AN4394 zugrunde. Bei dieser Konfiguration wird der SPV1050 im Boost-Modus betrieben, um mit der niedrigen Spannung der PV-Zelle (0,5 bis 2,5V) den LiPo-Akku bis 4,2V aufzuladen. Als Zelle wurde MPT2.4-21 von PowerFilm gewählt dank der kompakten Abmessungen (54mmx21mm). Die Solarzelle liefert bei voller Sonneneinstrahlung bis zu 2,4V und 16mA und eine Leistung von 0,035W. Die approximate Ladezeit eines vollständig entleerten Akkus beträgt damit $\frac{1500\text{mAh}}{16\text{mA}} = 93,75\text{h}$, was in der Praxis aufgrund von Wetterbedingungen und Ladeeffizienz wahrscheinlich eher 2-3 Wochen entspricht. Das klingt zwar nach wenig, aber der evt. jahrelange Betrieb rechtfertigt die Integration dieser Solarzelle.

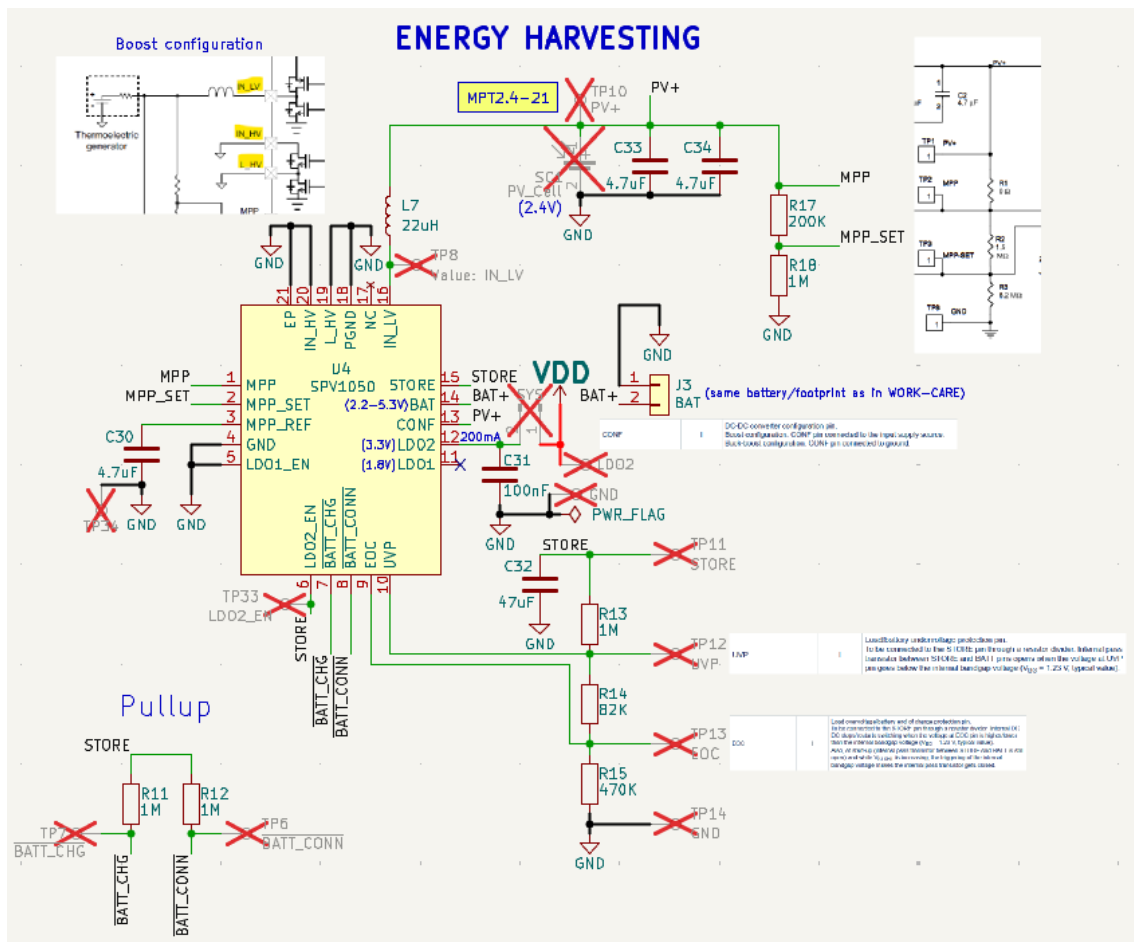


Abbildung 3.9: Energiegewinnung durch Solarzelle mit SPV1050

Der SPV1050 verfügt über einen Maximum Power Point Tracking (MPPT)-Algorithmus, der die optimale Last für die Solarzelle berechnet, um die Energieausbeute zu maximieren. Der Feedback für den MPPT-Algorithmus wird über den Spannungsteiler mit R17 und R18 bereitgestellt. Des Weiteren ist erwähnenswert, dass der SPV1050 intern auch zwei LDOs besitzt: LDO1 liefert 1,8V für RF-Schaltungen, wird jedoch hier nicht benutzt; LDO2 kann bis zu 200mA bei 3,3V ausgeben und könnte bei künftigen Iterationen sogar VDD versorgen, soweit das Stromprofil der einzelnen Komponenten bekannt ist.

Bezüglich der Batterie ist es wichtig, die Standzeit zu beobachten. Die Ladereglerchips verfolgen die Batteriespannung zwar zeitnah, aber allein diese gibt keine aussagekräftige Information, da eine Vielfalt anderer Faktoren wie Temperatur, Alterung und Entladungsrate die tatsächliche Kapazität beeinflussen können. Daher wird der Fuel-Gauge-Chip MAX17048 von Maxim Integrated eingesetzt, um den State of Charge (SoC) der Batterie zu verfolgen. Dieser schickt seine Daten über I2C an die MCU und benachrichtigt sie über die ALERT Leitung, sobald unter einen bestimmten Schwellenwert fällt.

viel geringeren Performance. Außerdem muss die im Kapitel 1 angesprochene Sendeleistungsbegrenzung von 14dBm in Europa beachtet werden.

All diese Überlegungen führten schließlich zum Design MB1720. Als Antenne hat sich nach der Recherche das Modell AEACAC053010-S868 von der Firma Abracon durchgesetzt mit seiner omnidirektionalen Strahlungscharakteristik, einem Gewinn von 2dBi und einer Impedanz von 50 Ohm. Bei künftigen Iterationen wird mit einer Chip-Antenne experimentiert, um die Kosten weiter zu senken.

Die vom Referenzdesign angepasste MCU-Schaltung für den ersten Prototyp sieht in ihrer Gesamtheit folgendermaßen aus:

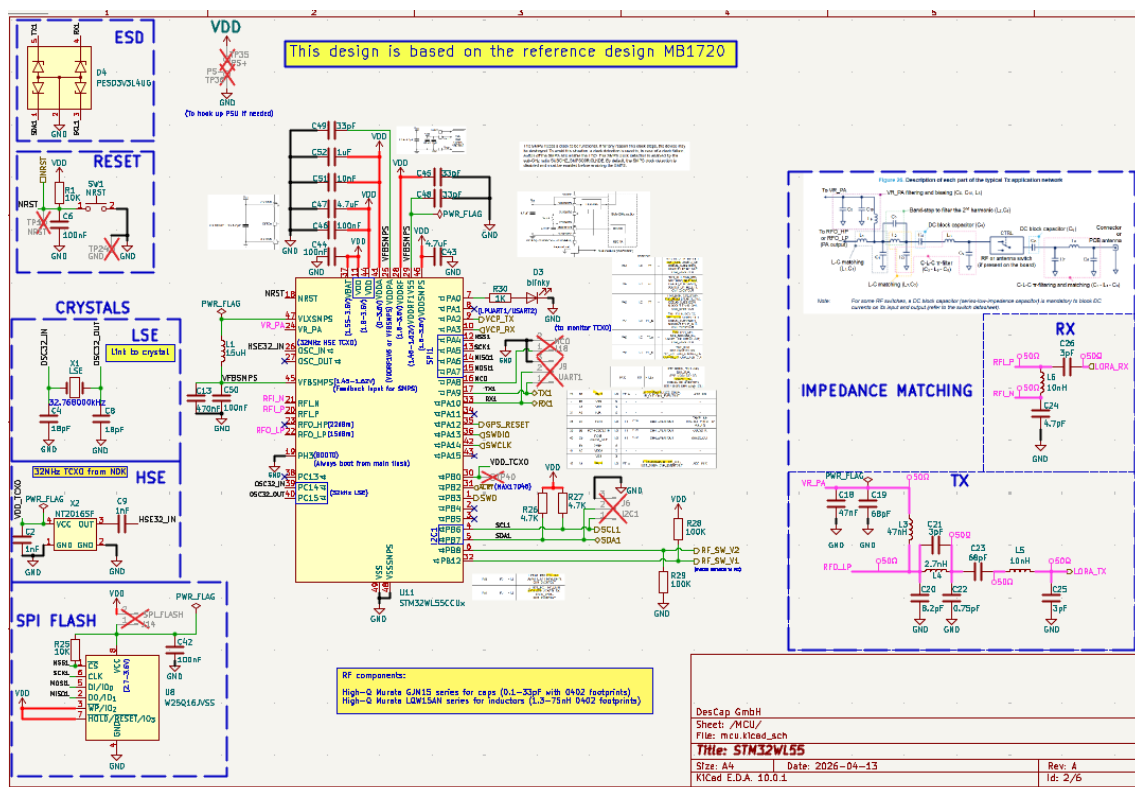


Abbildung 3.11: MCU-Schaltung mit STM32WL55

Taktversorgung

Der STM32WL55 benötigt zwei unabhängige Taktquellen: einen Hochfrequenz-Takt (High Speed External (HSE)) für den RF-Transceiver und einen Niederfrequenz-Takt (Low Speed External (LSE)) für den Echtzeittaktgeber (Real-Time Clock (RTC)).

Als HSE-Referenz wird ein 32 MHz-TCXO vom Typ **NDK NT2016SF** verwendet. Gegenüber einem passiven Quarzresonator bietet der TCXO eine deutlich höhere Frequenzstabilität: Während ein Standard-Quarz über den Betriebstemperaturbereich Abweichungen von $\pm 20\text{--}30$ ppm

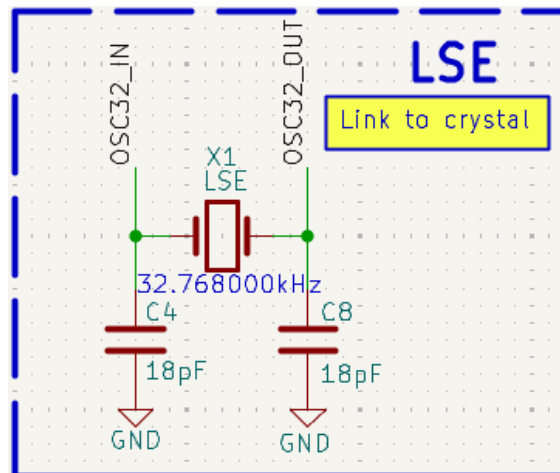


Abbildung 3.14: Taktversorgung mit LSE

Für den RTC ist ein passiver Quarzresonator mit der Nennfrequenz 32 768 Hz bestückt. Hierfür wurde der Typ FC-135 von Seiko Epson mit einer Nennlastkapazität von 12,5 pF ausgewählt. Die symmetrischen Lastkapazitäten C4 und C8 lassen sich durch folgende Formel bestimmen:

$$C_1 = C_2 = 2 \cdot (C_L - C_S),$$

wobei C_L die Nennlastkapazität des Quarzes und C_S die parasitäre Kapazität ist, die typischerweise zwischen 2 und 5 pF liegt. Unter der Annahme von $C_S = 3,5$ pF ergibt sich ein Wert von 18pF.

HF-Impedanzanpassung

Die HF-Impedanzanpassung stellt den sendeseitigen Leistungsübergang zwischen dem integrierenden Leistungsverstärker des STM32WL55 und der nachgelagerten RF-Schalterplatine sowie den empfangsseitigen Übergang von der Antenne zur internen Low Noise Amplifier (LNA) sicher. Sämtliche Bauteile dieser Sektion entstammen der Murata GJM15-Serie (Kondensatoren) und der Murata LQW15AN-Serie (Induktivitäten), da diese Komponenten im relevanten Frequenzbereich bei 868 MHz einen besonders hohen Gütefaktor ($Q > 100$) aufweisen und damit die Verluste im Anpassnetzwerk minimieren.

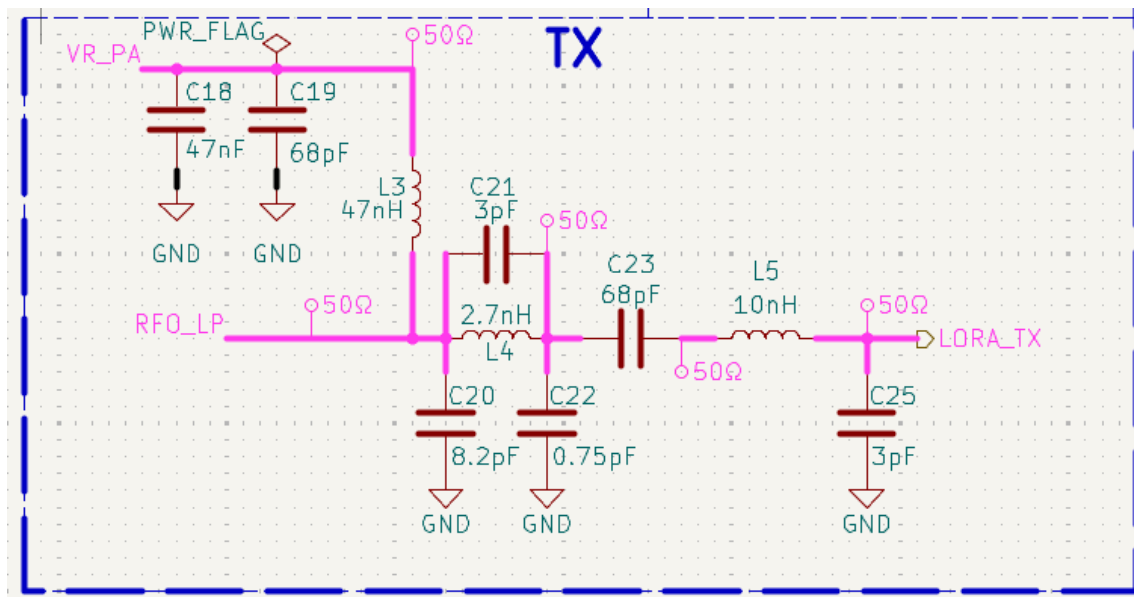


Abbildung 3.15: TX-Impedanzanpassung

Sendepfad (TX) Der Sendepfad führt vom Niederleistungsausgang RF0_LP (max 15 dBm) der MCU über das Anpassnetzwerk zum Signal LORA_TX, das zur Antennen-Teilschaltung weitergeleitet wird. Das Anpassnetzwerk besteht aus folgenden Elementen, deren Werte und Funktion in Anlehnung an den Applikationshinweis AN5406 von STMicroelectronics ausgelegt wurden:

- **C18** (47 nF): Entkoppelkondensator auf der VR_PA-Versorgungsleitung. Er filtert Versorgungsrauschen vom internen PA-Spannungsregler und ist gemäß AN5406 in unmittelbarer Nähe des VR_PA-Pins zu platzieren.
- **C19** (68 pF) und **L3** (47 nH): Bandsperre zur Unterdrückung der zweiten Harmonischen bei 1736 MHz. Die Kombination aus Parallelresonanzkreis wirkt als Sperrkreis bei $2f_0$ und reduziert so die Oberwellenemissionen, die für die Konformität mit EN 300 220 relevant sind.
- **L4** (2,7 nH) und **C20** (8,2 pF): L-Netzwerk zur Impedanztransformation. Sie überführen die komplexe Ausgangsimpedanz des PA in die Systemimpedanz von 50 Ω.
- **C21** (3 pF) und **C22** (0,75 pF): Feinabstimmung der Anpassung. Die sehr kleinen Kapazitätswerte sind empfindlich gegenüber parasitären Effekten und machen eine sorgfältige HF-gerechte Leiterbahnführung auf der Leiterkartenebene erforderlich.
- **C25** (3 pF): DC-Blockkondensator am Ausgang des Anpassnetzwerks, der Gleichspannung von der Antennenseite fernhält.

Der Hochleistungsausgang RF0_HP (bis 22 dBm) wird in dieser Designvariante nicht aktiv genutzt aufgrund der bereits erwähnten 14dBm-Grenze in der EU. Bei künftigen Iterationen könnte dieser Ausgang durchaus sinnvoll sein, um Leistungsverlusten im aktuellen Design entgegenzuwirken.

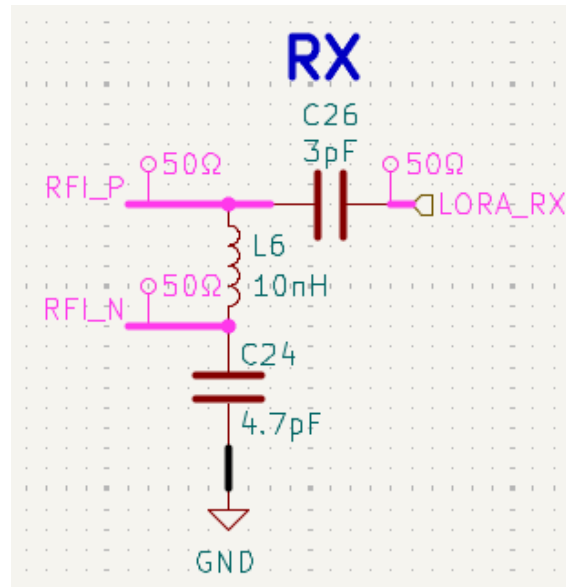


Abbildung 3.16: RX-Impedanzanpassung

Empfangspfad (RX) Der Empfangspfad führt vom Signal LORA_RX über ein Balun-Netzwerk zu den differentiellen Eingängen RFI_P und RFI_N des internen LNA. Das differentielle Eingangspaar erfordert eine Konvertierung des einseitigen (single-ended) 50 Ω -Signals der Antenne in ein symmetrisches Differenzsignal:

- **C26** (3 pF): Einkoppelkondensator am Eingang des Balun-Netzwerks. Blockiert Gleichspannung und dient als erstes Anpasselement.
- **L6** (10 nH): Serien-Induktivität im Signalpfad zu RFI_P. Bildet zusammen mit den Shunt-Kapazitäten den Impedanztransformator.
- **C24** (4,7 pF): Shunt-Kapazität gegen Masse im RFI_P-Pfad. Vervollständigt die Impedanztransformation zur differentiellen Eingangsimpedanz des LNA.
- **C23** (68 pF): Shunt-Kapazität im RFI_N-Pfad, symmetrisch zur GJM15-Kapazität im RFI_P-Zweig.
- **L5** (10 nH): Serien-Induktivität im RFI_N-Pfad zur Symmetrierung des Differenzsignals.

Interner Schaltregler (SMPS)

Der STM32WL55 integriert einen internen Abwärtswandler, der die digitale Kernversorgung aus der externen Versorgungsspannung (VDDSMPS) erzeugt. Der SMPS-Betrieb reduziert den Leistungsverbrauch im Aktivbetrieb gegenüber einer reinen LDO-Versorgung erheblich und ist der primäre Grund für den sehr niedrigen Ruhestrom von $1.08\ \mu\text{A}$ im Deep-Stop-Modus.

Die externe Beschaltung des SMPS umfasst:

- **L1** ($15\ \mu\text{H}$, MLZ2012M): Schaltinduktivität zwischen VDDSMPS und VLXSMPS. Der exakte Bauteilwert ist durch den internen Regler vorgegeben und darf nicht frei substituiert werden, da abweichende Induktivitätswerte die Stabilitätsreserve des Regelkreises beeinflussen.
- **C13** ($470\ \text{nF}$): Rückkopplungskapazität am VFBSMPS-Pin. Sie bestimmt die Ausgangsregelspannung des SMPS und ist ebenfalls ein stabilitätskritisches Bauteil.
- **C50** ($100\ \text{nF}$): Hochfrequenz-Bypass am VFBSMPS-Knoten.

Eine ausführliche Warnung im Schaltplan weist darauf hin, dass der SMPS eine Taktquelle benötigt: Fällt diese aus, schaltet der Regler automatisch in den LDO-Betrieb um. Da die SMPS-Taktdetektion standardmäßig deaktiviert ist, muss sie in der Firmware explizit aktiviert werden.

Spannungsversorgung und Entkopplung

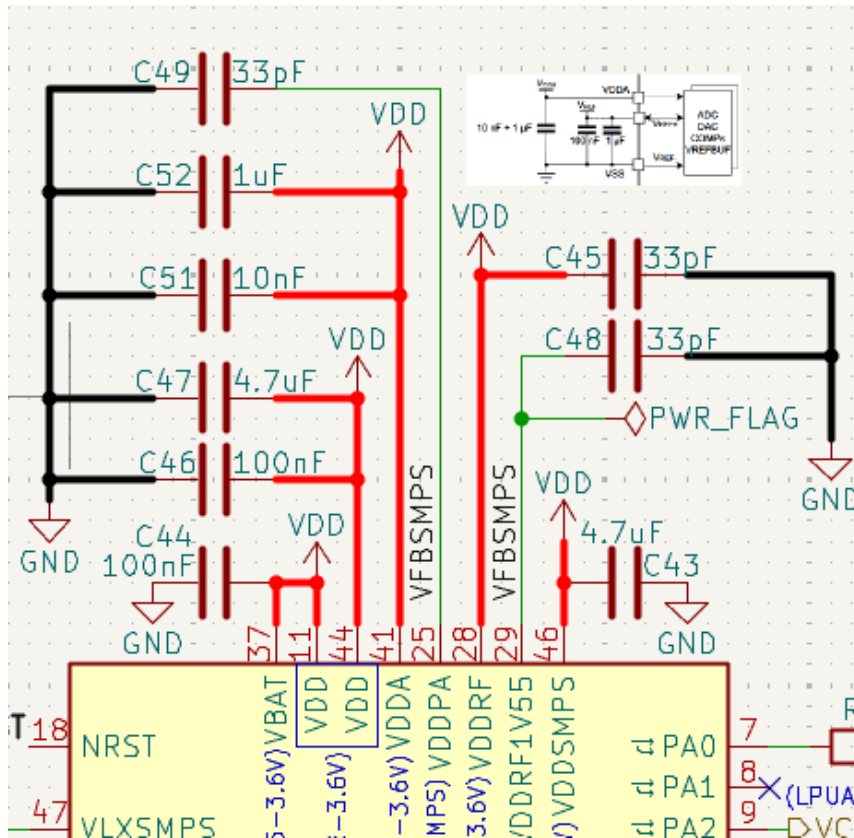


Abbildung 3.17: Entkopplungsschema der MCU

Die Entkopplungsstrategie folgt dem in der Referenz MB1720 empfohlenen Schema: Je VDD-Versorgungsdomäne wird eine Kombination aus einem Bulk-Kondensator ($4,7\ \mu\text{F}$) und einem lokalen HF-Bypass-Kondensator ($100\ \text{nF}$) eingesetzt. Diese Kondensatoren sind auf dem PCB in unmittelbarer Nähe der jeweiligen Versorgungspins zu platzieren, um den induktiven Anteil der Versorgungsleitungen zu minimieren.

Tabelle 3.3 fasst die Entkopplungskapazitäten und ihre Zuordnung zu den Versorgungsdomänen zusammen.

Tabelle 3.3: Entkopplungskondensatoren und ihre Funktionen

Bauteil	Wert	Domäne	Funktion
C43	4,7 μ F	VDD	Bulk-Entkopplung digitale Kernversorgung
C46	100 nF	VDD	HF-Bypass digitale Kernversorgung
C47	4,7 μ F	VDDSMPS	Bulk-Entkopplung SMPS-Eingang
C48	33 pF	VBAT / RTC	HF-Bypass RTC-Backup-Domäne
C49	33 pF	VDDRF1V55	HF-Bypass interner 1,55-V-LDO-Ausgang
C51	10 nF	VDDSMPS	Zwischenfrequenz-Bypass SMPS
C52	1 μ F	VDDSMPS	Mittlere Bulk-Entkopplung SMPS
C44	100 nF	VDDA	Bypass analoge Versorgungsdomäne (ADC)
C50	100 nF	VFBSMPS	HF-Bypass SMPS-Rückkopplungsknoten

Die analoge Versorgungsdomäne VDDA (Pin 41 des STM32WL55CCUx) ist mit C44 (100 nF) entkoppelt und über die Pulldown-Widerstände R28 und R29 (je 100 k Ω) referenziert. Diese Widerstände stellen sicher, dass VDDA und VREF+ im nicht bestückten Zustand definierte Pegel aufweisen und nicht floaten.

Reset-Schaltung

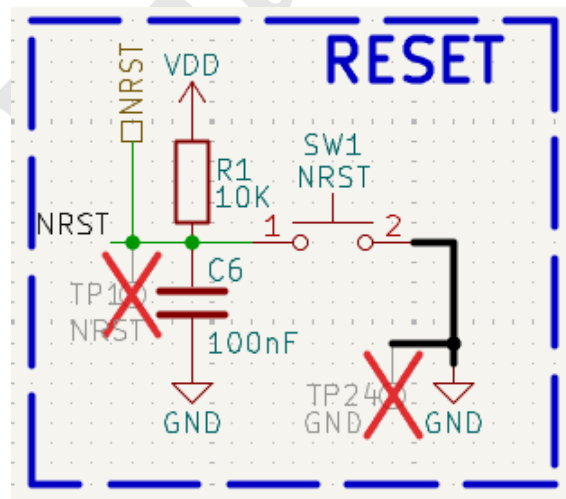


Abbildung 3.18: MCU-Reset-Schaltung

Die Reset-Schaltung besteht aus dem Pullup-Widerstand R1 (10 k Ω) zwischen VDD und NRST, dem manuellen Reset-Taster SW1 sowie dem Entkoppelkondensator C6 (100 nF). Der Widerstand R1 hält den Reset-Pin im Normalbetrieb auf High-Pegel, während SW1 einen manuellen Reset

durch Kurzschluss gegen Masse ermöglicht. C6 filtert kurze Spannungseinbrüche auf NRST und verhindert dadurch unbeabsichtigte Resets.

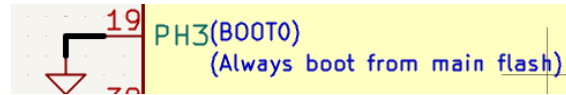


Abbildung 3.19: BOOT0-Konfiguration

BOOT0-Konfiguration Der STM32WL55 hat verschiedene Boot-Modi, die durch den Pegel am Pin BOOT0 (PH3) bestimmt werden. Dieser wird im Design der Einfachheit halber direkt auf Masse gezogen, sodass der Mikrocontroller beim Einschalten oder nach jedem Reset stets aus dem internen Flash-Speicher Code ausführt.

I2C-Schnittstelle und Pullup-Beschaltung

Die I2C1-Schnittstelle (SDA1, SCL1) verbindet die MCU mit dem Kraftstoffmessgerät MAX17048 sowie dem Gas-Sensor BME680. Die Pullup-Widerstände R26 und R27 (je 4.7 kΩ) ziehen beide Leitungen auf VDD. Für den I2C-Betrieb im Standard-Mode (100 kHz) ist dieser Wert unkritisch. Bei einem geplanten Übergang zu Fast Mode (400 kHz) wäre eine Reduktion auf 2.2 kΩ zu prüfen, da die RC-Zeitkonstante ($\tau = R \cdot C_{\text{Bus}}$) mit den geschätzten Busleitungskapazitäten von 50–100 pF bei 4.7 kΩ an die Grenze der zulässigen Anstiegszeit stößt.

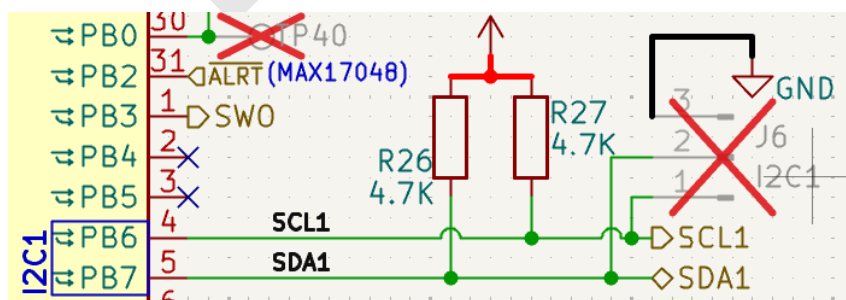


Abbildung 3.20: I2C-Schnittstelle

Der Header J6 (I2C1) erlaubt den direkten Anschluss eines Oszilloskops zur Visualisierung des I2C-Protokolls. Da alle Kanäle des Oszilloskops eine gemeinsame Masse haben, wird hier nur ein GND-Anschluss anstatt zwei für SCL und SDA vorgesehen.

ESD-Schutz

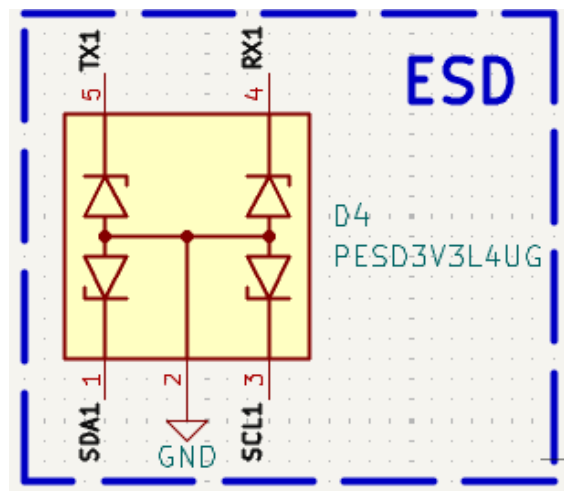


Abbildung 3.21: ESD-Schutz für Testheaders

Die Oszilloskop-Testheaders für I2C sowie UART (J6 und J9) können gefährliche elektrostatische Entladungen (ESD) von externen Messgeräten oder Bedienern auf die empfindlichen MCU-Pins übertragen. Aus diesem Grund wird eine Schutzdiodenarray (D4) vom Typ **PESD3V3L4UG** vorgesehen. Das Bauteil besteht aus vier Schutzdioden, welche die Überspannungsspitzen auf 3.3 V abklemmen. Die Dioden sind so angeordnet, dass sie sowohl positive als auch negative ESD-Impulse ableiten können.

SPI-Flash-Speicher

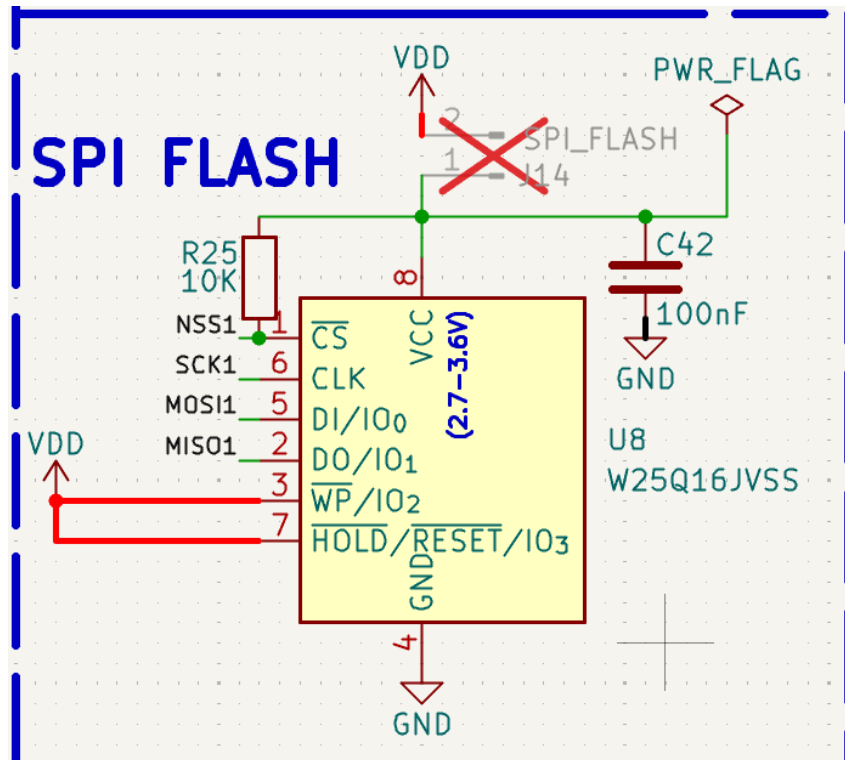


Abbildung 3.22: SPI-Flash-Speicher

Für die persistente Datenspeicherung, sei es für Debugmeldungen oder für Sensordaten, ist ein SPI-Flash-Speicher **W25Q16JVSS** (U8, 16 Mbit) bestückt. Das Bauteil wird über die SPI1-Peripherie der MCU angebunden (NSS1, SCK1, MOSI1, MISO1). Der bei Flash-Speichern übliche Limit von 100.000 Schreibzyklen sollte keine Hürde für den vorgesehenen Einsatz darstellen.

Jenseits des Prototypens kann der SPI-Flash-Speicher zusätzlich noch für drahtlose Firmware-Updates über das LoRaWAN-Netzwerk (FUOTA - Firmware Update Over The Air) genutzt werden. Da der interne Flash-Speicher des STM32WL55CCUx (256 kB) nicht ausreicht, um gleichzeitig die laufende Firmware und ein neu empfangenes Firmware-Image vorzuhalten, übernimmt der externe W25Q16JVSS (2 MB) die Rolle eines Zwischenspeichers.

Status-LED

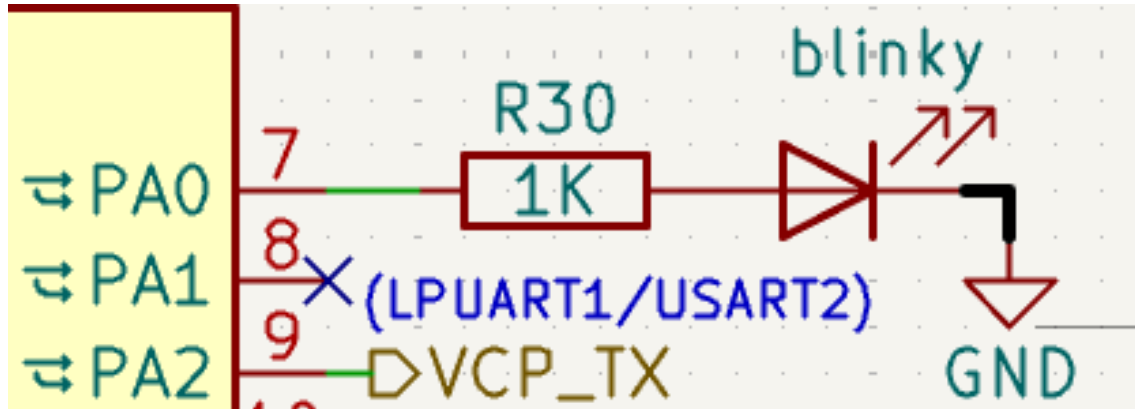


Abbildung 3.23: Status-LED

Eine Status-LED D3 (*blinky*) ist über den Vorwiderstand R30 (1 k Ω) an den GPIO-Pin PA0 angebunden. Sie dient beim ersten Prototypen hauptsächlich der schnellen Überprüfung des Flashprozesses: nach Erhalt der PCBs wird ein einfaches blinky-Programm in die MCU eingeschleust, das die LED in einem festen Intervall blinken lässt. Ist dies erst erfolgreich, kann die eigentliche Firmware-Entwicklung mit der Sensoranbindung und LoRaWAN-Funktionalität implementiert werden.

Zusammenfassung der Signalnetze

Tabelle 3.4 gibt einen Überblick über die wichtigsten Signalnetze, die die MCU-Teilschaltung mit den übrigen Teilschaltungen des Prototyps verbinden.

Tabelle 3.4: Externe Signalnetze der MCU-Teilschaltung

Signal	Zielschaltung	Funktion
LORA_TX	Antenne (Sheet 5)	Sendepfad 868 MHz zum RF-Schalter
LORA_RX	Antenne (Sheet 5)	Empfangspfad 868 MHz vom RF-Schalter
RF_SW_V1/V2	Antenne (Sheet 5)	Steuersignale AS179-92LF RF-Schalter
SDA1 / SCL1	GPS/Sensoren (Sheet 6)	I2C1-Bus (BME680, MAX17048)
GPS_RESET	GPS/Sensoren (Sheet 6)	Aktiv-Low Reset Quectel L80-R
TX1 / RX1	GPS/Sensoren (Sheet 6)	UART1 zu GPS-Modul L80-R
SCK1/MOSI1/MISO1/NSS1	MCU-intern	SPI1 zu Flash W25Q16JVSS
SWDIO / SWCLK / SW0 / NRST	Prog. (Sheet 3)	SWD-Debugschnittstelle
VCP_TX / VCP_RX	Prog. (Sheet 3)	Virtueller COM-Port (STLink V3)
SDA_FG / SCL_FG (= SDA1/SCL1)	Power (Sheet 4)	I2C1 zu Fuel Gauge MAX17048
Ä	Power (Sheet 4)	Low-Aktiv Interrupt MAX17048
ÄBATT_CHG / ÄBATT_CONN	Power (Sheet 4)	Ladestatus BQ24074

3.3.2 PCB-Layout

Aufbau

Für das Design wurde ein *4-Layer-Stackup* (vierlagige Leiterplatte) gewählt. Die zwei geläufigsten Anordnungen bei diesem Aufbau sind SGN-GND-GND-SGN und SGN-GND-PWR-SGN, und es wurde für das Letztere entschieden aus persönlichem Vorzug. Die vier Lagen haben folgende Funktionen:

- **Top Layer (L1):** Platzierung der Bauteile und primäre Signalleitungen (inklusive der HF-Leitungen)
- **Inner Layer 1 (L2):** Durchgehende Massenfläche (Ground Plane) als Potenzial

- **Inner Layer 2 (L3):** Stromversorgungsbussysteme (Power Plane) zur niederimpedanten Verteilung der Betriebsspannung (3,3V).
- **Bottom Layer (L4):** Sekundäre Signalleitungen

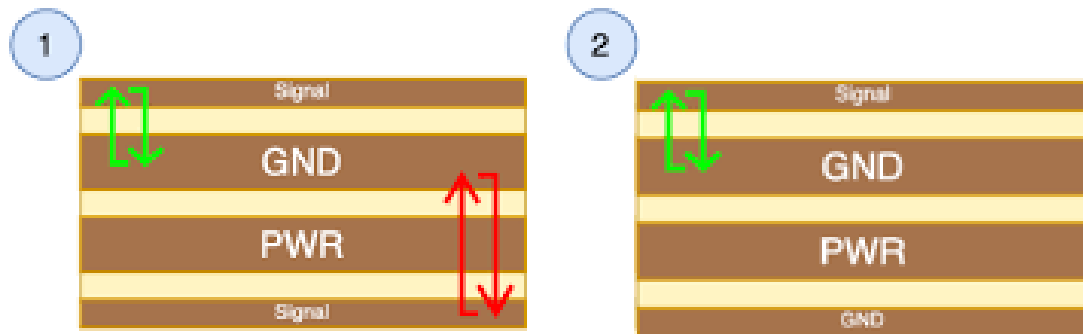


Abbildung 3.24: PCB-Stackup mit vier Lagen: Top (L1), Inner 1 (L2), Inner 2 (L3), Bottom (L4)

Dieser Aufbau kostet im Vergleich zu einem einfachen zweilagigen Design zwar geringfügig mehr bei JLCPCB, bietet aber entscheidende Vorteile für Hochfrequenzanwendungen. Die durchgehende Massenfläche L2 dient als Rückstrompfad für HF-Signale, sodass die elektrischen Feldlinien sich nicht unkontrolliert ausbreiten und andere Komponenten stören. Der geringe L1-L2-Abstand (0,2mm) beim 4-Layer-Stackup sorgt auch für kleinere Stromschleifen. Beim 2-Layer hingegen beträgt dieser Abstand 1,5mm, dadurch würde diese SGN-GND-Schleifen in diesem Fall 7,5-mal so stark Störungen abstrahlen/empfangen.

Aus Abbildung 3.24 ist ersichtlich, dass die Signallagen L1 und L4 bezüglich GND (L2) asymmetrisch sind. L1 ist viel näher an GND und soll daher für alle empfindlichen Leitungen (dazu RF-Leiterbahnen) verwendet werden. L4 hingegen ist weiter von GND entfernt und eignet sich daher für weniger kritische Signale, wie z.B. die I2C-Leitungen zu den Sensoren.

Auf L3 ist die ebenfalls durchgehende 3,3V Versorgungsschicht untergebracht. Aus EMV-technischer Sicht ist es jedoch suboptimal, da an den PCB-Kanten gestrahlt werden kann. Der Anwendungshinweis AN5407 von ST empfiehlt daher, Abstand von der Power Plane zum Boardrand zu lassen und einen GND-Ring vorzusehen, welcher über Via-Stitching mit L2 verbunden werden soll (Abbildung 3.25)

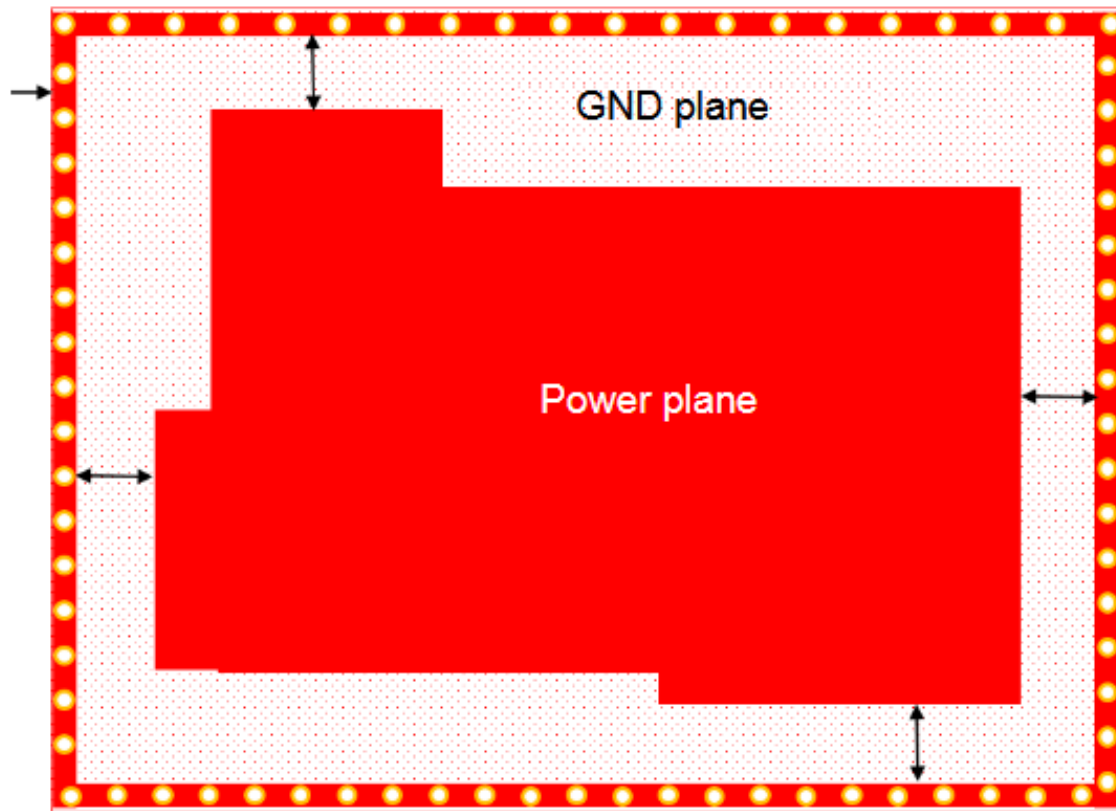


Abbildung 3.25: Optimale Power-Plane-Auslegung laut AN5407

Aufgrund dessen, dass JLCPCB eine teurere Preisklasse für Leiterplatten größer als 100 mmx100 mm ansetzt, wurde das Design trotz der Vielzahl an Modulen möglichst kompakt gehalten. Die finalen Dimensionen betragen 90x100 mm.

Impedanzkontrolle

Sämtliche RF-Leiterbahnen (lilafarbene Verbindungen im Schaltplan) müssen auf eine charakteristische Impedanz von $50\ \Omega$ ausgelegt werden, um die Leistungsverluste durch Signalreflexionen an den Übergängen (bswp. zur MCU oder Antenne) zu vermeiden. Die Berechnung der Leiterbahngeometrie erfolgt mit einem Impedanzrechner, der die Stackup-Parameter (Lagenabstände, Dielektrizitätskonstante) sowie die Leiterbahnbreite und -dicke berücksichtigt.

Abbildung 3.26: Impedanzberechnung in KiCad

KiCad verfügt über einen eingebauten Rechner und die einzutragenden Informationen (gelb markiert) ergeben eine Leiterbahnbreite $W = 0.35 \text{ mm}$. Leichte Abweichungen von der 50Ω können allerdings entstehen aufgrund von bswp. mechanischen Toleranzen oder Varianz der Dielektrizitätskonstante ϵ_r des verwendeten FR4-Materials.

Impedanzkritische Verbindungen müssen knickfrei sein. KiCad stellt diverse Erweiterungen bereits, mit denen eckige Übergangsstellen in Verrundungen verwandelt werden können. Problematisch bleiben allerdings die Übergänge von Leiterbahnen zu Pads von SMD-Komponenten, da diese meist viel größer sind als die berechnete 0.35 mm . Daher werden alle RF-SMD-Bauteile in der Größe 0402 (0.5 mm Padbreite) gewählt, und die 0.15 mm Differenz wird durch ein Teardrop-Muster überbrückt werden.

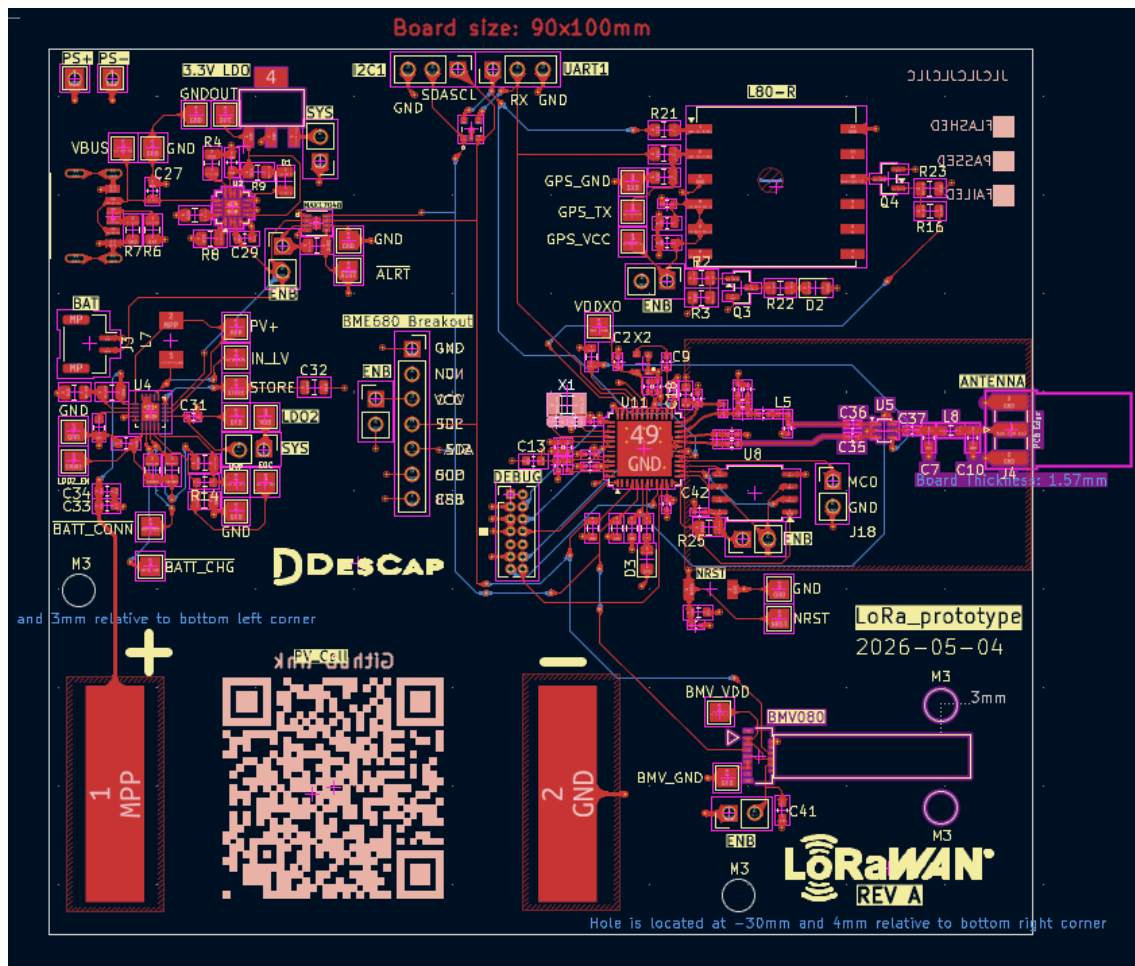


Abbildung 3.28: 2D-Ansicht der fertigen Leiterplatte

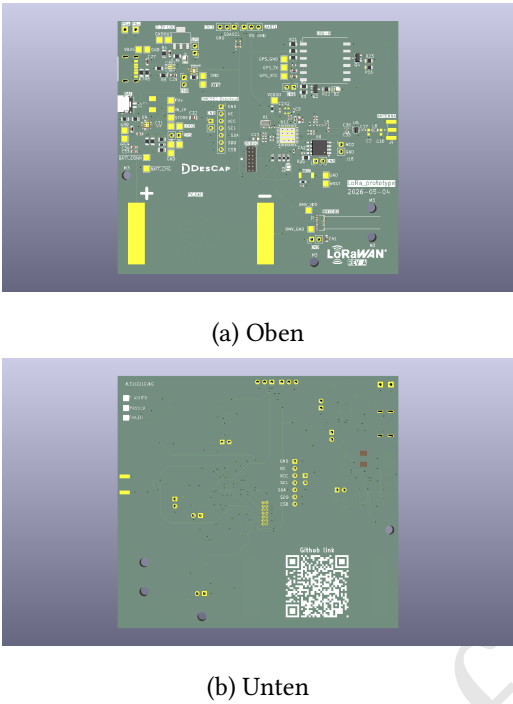


Abbildung 3.29: 3D-Ansichten der fertigen PCB

3.3.3 Fertigung und Bestückung

Es wurden insgesamt 5 PCBs bestellt bei JLCPCB, wovon nur 2 bestückt werden, um die Bauteilkosten zu reduzieren. Spezielle Komponenten, die entweder nicht bei JLCPCB bzw. LCSC erhältlich sind oder deren Bestückung die Kosten in die Höhe treiben, mussten zusätzlich über Mouser angeschafft werden. Dazu zählen z.B die Antenne, der dazugehörige SMA-Anschluss, die Solarzelle sowie THT-Pin-Headers, die händisch gelötet werden müssen. Eine approximative Kostenübersicht setzt sich zusammen aus:

Tabelle 3.5: Kostenaufstellung pro PCB

Posten	Kosten [€]
PCB-Fertigung (\$1,40)	1,29
PCBA-Bestückung & interne Komponenten (\$79,00)	72,68
SMA-Antennenanschluss (\$2,50)	2,30
868 MHz-Antenne	5,66
Solarzelle	4,63
Gesamtkosten pro Stück	86,56

Die 5 PCBs sind etwa zwei Wochen nach Bestellaufgabe in Berlin eingetroffen. **Der kom-**

platte Designprozess (von Schematic bis PCB-Erhalt) des ersten Prototyps hat also um die 6 Wochen gedauert.

3.4 Firmware-Entwicklung

3.4.1 Wahl des Betriebssystems: Zephyr-RTOS

Für die Implementierung der Firmware auf dem STM32WL55 wurde die Entscheidung getroffen, anstelle von Bare-Metal oder HAL-Verwendung (Hardware Abstraction Layer von ST Microelectronics) ein Echtzeitbetriebssystem (Real-Time Operating System - RTOS) einzusetzen. Der Bare-Metal-Ansatz, bei dem jedes Register bitweise manuell eingestellt werden muss, bietet zwar die maximale Kontrolle über die Hardware, ist aber im eingeschränkten gegebenen Zeitrahmen dieser Bachelorarbeit nicht realisierbar aufgrund der Komplexität der geplanten Funktionen. Bei einem MCU-Wechsel, selbst innerhalb der STM32WL-Familie, müsste die Firmware komplett neugeschrieben und Datenblätter neugelesen werden, was keine nachhaltige Entwicklungsstrategie darstellt.

Das letzte Problem lässt sich durch HAL beheben, da es eine einheitliche API über verschiedene STM32-Modelle hinweg schafft und somit registerspezifische Details wegabstrahiert sind. Ähnlich wie beim Bare-Metal-Ansatz befindet sich der Hauptcode in einer `while(1)`-Superschleife und wird sequenziell abgearbeitet. Eine reine HAL-Implementierung für komplexe Anwendungen (hohe Peripherienanzahl bswp.) könnte schnell zu gegenseitigen Ressourcenblockaden führen. Gerade das LoRaWAN-Protokoll stellt sehr stringente Timing-Anforderungen, wie in Kapitel 2 beschrieben, und erfordert eine präzise Koordination von Sende- und Empfangsfenstern. Es sind also Multitasking-Features unbedingt notwendig, die ein RTOS zur Verfügung stellen. Ein RTOS hat an erster Stelle dafür zu sorgen, dass zeitkritische Tasks innerhalb einer vordefinierten Frist gestartet und beendet werden (daher die Bezeichnung *real-time*). Ein RTOS ist also zeitlich deterministisch, im Gegensatz zu konventionellen Betriebssystemen wie Windows oder Linux, die nicht garantieren können, wann genau eine Task ausgeführt wird. Um diesen Determinismus zu gewährleisten, macht sich der RTOS-Kernel das Konzept von **Preemptive Scheduling** zunutze: dabei bekommt jede Task eine Priorität zugewiesen, und der Scheduler sorgt dafür, dass immer die Task mit der höchsten Priorität ausgeführt wird. Eine laufende Task mit niedriger Priorität wird also unterbrochen (*präemptiert*), sobald eine höherprioritäre Task bereit ist, ausgeführt zu werden.

Obwohl der Markt für eingebettete Echtzeitbetriebssysteme eine große Vielfalt an etablierten Lösungen bietet, darunter das weit verbreitete FreeRTOS oder die herstellereigene Middleware

STM32CubeOS, fiel die Wahl für dieses Projekt gezielt auf Zephyr RTOS. Der entscheidende Vorteil von Zephyr liegt in seiner modernen, modular aufgebauten Architektur, die speziell für vernetzte, ressourcenbeschränkte IoT-Knoten optimiert ist.

Im Gegensatz zu FreeRTOS, das primär einen minimalen Kernel bereitstellt und die manuelle Integration von Netzwerk-Stacks erfordert, bringt Zephyr den industriell erprobten Semtech-LoRaWAN-Stack (LoRaMac-node) als natives Subsystem direkt mit. Dies reduziert nicht nur die Fehleranfälligkeit bei der Implementierung drastisch, sondern stellt auch die Einhaltung regionaler Funkrestriktionen (wie des Duty-Cycles im EU868-Band) auf Betriebssystemebene sicher. Ein weiteres Kernargument für Zephyr ist das Board-Agnostic-Prinzip. Durch die strikte Abstraktion der Hardware über das Devicetree-System ist die Firmware nicht monolithisch an einen spezifischen Mikrocontroller gebunden. Sollte im Produktlebenszyklus ein Wechsel des System-on-Chip notwendig werden, bleibt der Applikationscode unverändert; lediglich das Hardware-Overlay muss an die neue Pinbelegung angepasst werden. Diese inhärente Portierbarkeit, kombiniert mit einer reichhaltigen Bibliothek an integrierten Peripherietreibern für die verwendete Sensorik (wie den BME680), qualifiziert Zephyr RTOS als zukunftsichere und nachhaltige Entwicklungsplattform für den hier entwickelten Prototyp.

3.4.2 Zephyr-Architektur: Devicetree, KConfig, Treibermodelle

Die Besonderheit von Zephyr RTOS liegt in seiner strikten und konsequenten Trennung zwischen Hardwarebeschreibung, Softwarekonfiguration und dem eigentlichen Applikationsquellcode. Diese dreigeteilte Architektur spiegelt sich im Zusammenspiel von Devicetree, Kconfig und dem generischen Treibermodell wider und bildet das Fundament für die im vorigen Abschnitt beschriebene Plattformunabhängigkeit.

Devicetree zur Hardware-Beschreibung

Der Devicetree (DT) ist eine Baumdatenstruktur, die die physische Hardware des Zielsystems abstrahiert abbildet. Ursprünglich aus dem Linux-Kernel übernommen, beschreibt der DT alle Komponenten des Mikrocontrollers, von den CPU-Kernen über die Speicherbereiche (Flash, RAM) bis hin zu den Peripherieschnittstellen (I²C, SPI, UART) und den daran angeschlossenen externen Komponenten (Sensoren, Funkmodule).

Wie der Begriff „Baum“ schon suggeriert, werden sämtliche Hardware-Bausteine durch sogenannte *Nodes* (Knoten) dargestellt, die sich einer hierarchischen Struktur unterordnen. Jeder Node kann wiederum *Properties* (Eigenschaften) besitzen, welche in Form von Schlüssel-Wert-Paaren angegeben werden. Ein kurzer DT-Ausschnitt in Listing 3.1 hilft, die Syntax zu verdeut-

lichen:

Listing 3.1: Zephyr Devicetree-Syntax

```
1 / {
2     soc {
3         i2c0: i2c@40003000 {
4             compatible = "vendor,i2c-controller";
5             reg = <0x40003000 0x400>;
6             status = "okay";
7         };
8         uart0: uart@40002000 {
9             compatible = "vendor,uart-controller";
10            reg = <0x40002000 0x200>;
11            status = "okay";
12        };
13    };
14};
```

Der Schrägstrich `/` in Zeile 1 repräsentiert den sogenannten Root-Node, der in jedem DT genau einmal vorkommen muss. Er bildet die oberste Ebene des gesamten Hardware-Baums, von dem alle weiteren internen und externen Komponenten als Child-Nodes abzweigen. In Zeile 3 `i2c0: i2c@40003000` wird ein Peripherie-Node definiert. Der vordere Teil `i2c0` ist ein frei wählbares Label, das im späteren Applikationscode oder in Overlays als Kurzreferenz genutzt werden kann. Der Teil nach dem Doppelpunkt, `i2c@40003000`, spezifiziert den eigentlichen Nodennamen, gefolgt von der Unit-Address (hier die hexadezimale Basisadresse des I2C-Controllers im Speicher des Mikrocontrollers).

Die `compatible`-Property in Zeile 4 ist besonders wichtig. Es handelt sich dabei um eine Zeichenkette im Format `"Hersteller,Modell"`, die als eindeutiger Schlüssel für das Treiber-Matching dient, über den der Kernel erkennt, welcher hardwarespezifische C-Treiber für diese Komponente geladen werden muss. Die `reg`-Property in Zeile 5 gibt den Speicherbereich an, der dem I2C-Controller zugeordnet ist, während die `status` in Zeile 6 den Betriebszustand des Controllers beschreibt. Der `uart`-Node wird genauso aufgebaut wie der `i2c`-Node.

Abbildung 3.30 zeigt den daraus resultierenden Baum. Diese im primären Devicetree definierte Struktur beschreibt standardmäßig jedoch nur die vom Chiphersteller fest integrierten On-Chip-Peripherieblöcke. Da eine reale eingebettete Anwendung in der Praxis um externe Sensoren und Aktoren auf der Leiterplatte erweitert wird, stellt sich die Frage, wie diese systemspezifische Hardwaretopologie in die bestehende Baumstruktur integriert werden kann.

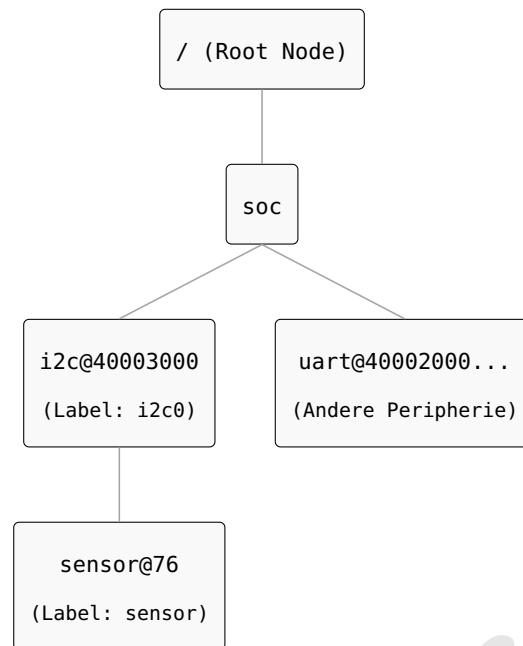


Abbildung 3.30: Hierarchische Baumstruktur des beispielhaften Devicetrees.

Für den Entwickler ist genau an dieser Schnittstelle das Konzept der **DT-Overlays** von zentraler Bedeutung. Während die Basis-Hardware des STM32WL55-Boards in den vom Betriebssystem bereitgestellten Standard-Dateien fest definiert ist, erlaubt es eine Overlay-Datei (mit Endung `.overlay`), projektspezifische Änderungen vorzunehmen, ohne den Kernel-Quellcode zu modifizieren. In dieser Arbeit wird das Overlay genutzt, um dem Zephyr-Kernel mitzuteilen, auf welche GPIO-Pins die serielle Schnittstelle des GPS-Empfängers abgebildet ist und unter welcher hexadezimalen Adresse der Umweltsensor BME680 auf dem I2C-Bus zu finden ist. Zur Kompilierzeit parst das Build-System diese Textdateien und generiert daraus C-Präprozessor-Makros, über die die Firmware direkt auf die Hardwarekonfiguration zugreifen kann.

KConfig zur Software-Konfiguration

Während der Devicetree definiert, welche Hardware existiert, steuert das Kconfig-System, welche Softwarefeatures und Treiber in das finale Firmware-Image kompiliert werden. Über die Konfigurationsdatei `prj.conf` kann der Entwickler den Funktionsumfang des Betriebssystems granular bestimmen.

Dies erlaubt ein hocheffizientes Ressourcenmanagement auf dem STM32WL55: Module, die für den spezifischen Anwendungsfall benötigt werden (wie z.B. GPIO-Treiber), müssen in `prj.conf` explizit aktiviert werden (`CONFIG_GPIO=y`). Das Kconfig-System stellt somit sicher, dass das resultierende Image, welches auf dem Mikrokontroller hochgeladen wird, einen minimalen Footprint aufweist und die Speicherrestriktionen einhält.

Treibermodelle als Brücke zwischen Hardware und Software

Das Bindeglied zwischen den Hardware-Strukturen des Devicetrees und den Software-Schaltern von Kconfig bildet das generische Treibermodell von Zephyr. Es stellt standardisierte APIs bereit, die plattform- und herstellerunabhängig operieren.

Anstatt Peripherie über proprietäre Registerbefehle des jeweiligen Chipherstellers anzusprechen, interagiert die Anwendungsschicht ausschließlich mit abstrakten Subsystemen (wie der `sensor_driver_api` oder `uart_driver_api`). Für die Anwendungslogik ist es dadurch irrelevant, welcher konkrete Sensor- oder Bus-Typ physisch verbaut ist. Ein Wechsel des Hardware-Bausteins erfordert lediglich eine Anpassung des `compatible`-Eintrags im DT; der Applikationscode greift über identische Funktionsaufrufe auf die Daten zu, da der Kernel den hardware-spezifischen Treiber vollautomatisch unter der standardisierten API kapselt.

3.4.3 LoRaWAN-Netzwerkanbindung

Mit dem Grundwissen über Zephyr im vorangegangenen Abschnitt ist es nun möglich, die Entwicklung des Anwendungscodes zu starten. Als Allererstes muss das PCB mit einem LoRaWAN-Netzwerk verbunden werden. Da der Aufbau und die Inbetriebnahme eines Privatnetzwerks den zeitlichen Rahmen dieser Arbeit sprengen würde, wird auf das öffentliche The Things Network (TTN) zurückgegriffen.

Wie aus Kapitel 2 bekannt, sind drei Informationen für die Registrierung eines Endgeräts im Netzwerk erforderlich: DevEUI, AppKey und JoinEUI. Zum Extrahieren der DevEUI wird Zephyr das Subsystem `hwinfo` zur Verfügung gestellt. TTN übernimmt die Rolle des Join-Servers und generiert selber den AppKey für unsere Applikation. Da kein externer Join-Server nötig ist, wird die JoinEUI als Nullvektor `00 00 00 00 00 00 00 00 00` definiert.

The screenshot shows a registration form for a new end device in the LoRaWAN network. It is divided into two main sections: 'General information' and 'Activation information'.

General information

End device ID	rev-a-1
Frequency plan	Europe 863-870 MHz (SF9 for RX2 - recommended)
LoRaWAN version	LoRaWAN Specification 1.0.3
Regional Parameters version	RP001 Regional Parameters 1.0.3 revision A
Created at	Jun 9, 2026 11:40:00

Activation information

AppEUI	00 00 00 00 00 00 00 00
DevEUI	00 80 E1 15 06 92 66 73
AppKey	3A C3 C1 95 56 82 07 3B 3C D...

Abbildung 3.31: Registrierung eines neuen Endgeräts im LoRaWAN-Netzwerk bei The Things Network

Zur Aktivierung mit dem OTAA-Verfahren wird der LoRaWAN `class_a` Beispielcode von Zephyr als Basis verwendet. In Zeile 95 des Codes wird auf ein Problem hingewiesen:

Listing 3.2: Statischer DevNonce-Ansatz im originalen Zephyr-Beispielcode

```

86 // ...
87
88 join_cfg.mode = LORAWAN_ACT_OTAA;
89 join_cfg.dev_eui = dev_eui64;
90 join_cfg.otaa.join_eui = join_eui;
91 join_cfg.otaa.app_key = app_key;
92 join_cfg.otaa.nwk_key = app_key;
93 join_cfg.otaa.dev_nonce = 0u;
94
95 // ...

```

Die `dev_nonce` muss gemäß LoRaWAN-Join-Spezifikationen bei jedem Netzwerk-Join inkrementiert werden, sie wird hier allerdings statisch auf `0u` festgelegt. Das führt dazu, dass mit diesem Beispielcode maximal ein Join zulässig ist. Der Code muss an dieser Stelle also mit einem besseren Handhabungsmechanismus bezogen auf `dev_nonce` ergänzt werden.

Listing 3.3: Persistente Speicherung und Inkrementierung des DevNonce

```
1 #include <zephyr/settings/settings.h>
2
3 // ...
4
5 int main(void)
6 {
7     // ...
8     // Initialisierung des Settings-Subsystems und Laden des DevNonce aus dem Flash-
9     // Speicher
10    ret = settings_subsys_init();
11    ret = settings_register(&lorawan_settings_handler);
12    ret = settings_load();
13    // ...
14    // Versucht solange, bis Join-Request erfolgreich ist
15    while(1) {
16        ret = lorawan_join(&join_cfg);
17        if(ret == 0) {
18            break;
19        }
20        k_msleep(20000);
21    }
22    // Inkrementierung und persistente Rücksicherung des Zählers
23    ttn_dev_nonce++;
24    ret = settings_save_one("lorawan/devnonce", &ttn_dev_nonce, sizeof(ttn_dev_nonce)
25    );
26    // ...
27    // Hauptschleife
28    while(1) {
29        // ...
30    }
31 }
```

```

30
31 // ...

```

In Zeilen 9 bis 11 im Listing 3.3 wird der im Flash-Speicher hinterlegte Wert auf die Variable `dev_nonce` übertragen. Das Programm versucht anschließend, unter Verwendung der neugeladenen `dev_nonce` sowie DevEUI, JoinEUI und AppKey (diese werden im Struct `join_cfg` zusammengefasst) dem Netzwerk beizutreten. Schlägt der Versuch fehl, wird eine Wartezeit von 20 Sekunden zum Nächstversuch eingehalten, um nicht gegen Duty-Cycle-Beschränkungen zu verstoßen. Ein erfolgreicher Verbindungsaufbau bewirkt, dass in Zeilen 21 und 22 die `dev_nonce` inkrementiert und wiederum im Flash zurückgespeichert wird. Somit kann das Board bei Bedarf gültige Rejoin-Anfragen senden, etwa beim Verbindungsabbruch oder nach einem Reset-Ereignis.

Zu guter Letzt müssen beim Join-Verfahren die TX- und RX-Fenster zeitlich abgestimmt sowie koordiniert werden, weswegen der LoRaWAN-Stack Auskünfte über die Pinverbindungen des RF-Schalters braucht. Da muss die Hardware-Beschreibung in der `.overlay`-Datei entsprechend erweitert werden, wie in Zeilen 5 und 6 im Listing 3.4 gezeigt wird.

Listing 3.4: Devicetree-Overlay zur Konfiguration des RF-Schalters

```

1 &subghzspi {
2     status = "okay";
3     lora: radio@0 {
4         status = "okay";
5         tx-enable-gpios = <&gpio 8 GPIO_ACTIVE_HIGH>;
6         rx-enable-gpios = <&gpio 12 GPIO_ACTIVE_HIGH>;
7     };
8 };

```

3.4.4 Integration von Peripherien

Nachdem die LoRaWAN-Netzwerkanbindung erfolgreich etabliert war, wurde im nächsten Schritt die Anbindung der Sensorik realisiert. Aus Vereinfachungsgründen wurden für den vorliegenden Prototyp ausschließlich der Gassensor BME680 sowie das GPS-Modul L80-R integriert; auf den Partikelsensors BMV080 wurde verzichtet, da dieser laut Schaltplan (vgl. Abschnitt ??) am selben I2C-Bus wie der BME680 betrieben wird und eine gleichzeitige Inbetriebnahme beider Sensoren den Rahmen dieser Arbeit nicht zusätzlich gerechtfertigt hätte.

Für den Umgebungssensor BME680 stellt Zephyr einen nativen Hardware-Treiber bereit, welcher über die Property `compatible = "bosch,bme680"` im Devicetree-Overlay referenziert

wird. Der Sensor ist theoretisch in der Lage, Umgebungstemperatur, relative Luftfeuchtigkeit, den Luftdruck sowie flüchtige organische Gaskonzentrationen simultan zu erfassen. Im Rahmen dieses Proof-of-Concept wird jedoch primär die Temperatur abgefragt und an das LoRaWAN-Netzwerk übermittelt. Die Konvertierung der sensorinternen Rohdaten erfolgt hierbei zunächst in ASCII-Zeichenketten. Dies erhöht zwar die resultierende Payload-Größe und consequenterweise die Übertragungszeit, erleichtert jedoch die Evaluierung der LoRaWAN-Nachrichten.

Das GPS-Modul gibt die geografischen Positionsdaten standardmäßig über das serielle NMEA-0183-Protokoll via UART aus. Aufgrund dieser Standardisierung ist kein herstellerspezifischer Treiber notwendig; der im Zephyr-Kernel integrierte, generische NMEA-Treiber ist für den Anwendungszweck vollständig ausreichend. Die Akquisition der Positionsdaten samt den dazugehörigen Zeitstempeln erfolgt in einem separaten Software-Thread parallel zur Sensordatenerfassung. Diese entkoppelte Task-Struktur stellt sicher, dass die Geokoordinaten kontinuierlich aktualisiert werden, um im Zuge der späteren Feldtests empirische Aussagen über die Antennenreichweite sowie die TTN-Netzwerkabdeckung treffen zu können.

3.5 Ergebnisse

In diesem Kapitel werden die empirischen Messergebnisse des entwickelten LoRaWAN-Prototyps präsentiert und dokumentiert. Die Evaluierung gliedert sich in die funktionale Überprüfung der TTN-Infrastruktur, die kartografische Analyse der Funkreichweite im Berliner Stadtgebiet sowie die messtechnische Bestimmung des elektrischen Energiebedarfs im Laborbetrieb.

3.5.1 Validierung der Netzwerkanbindung

Vor der Durchführung der großflächigen Feldversuche im urbanen Raum wurde die grundlegende Funktionalität der Kommunikationsarchitektur verifiziert. Der Fokus lag hierbei auf der fehlerfreien Initiierung des OTAA-Aktivierungsverfahrens sowie der Korrektheit der empfangenen Datenpakete auf der TTN-Serverseite der Applikation.

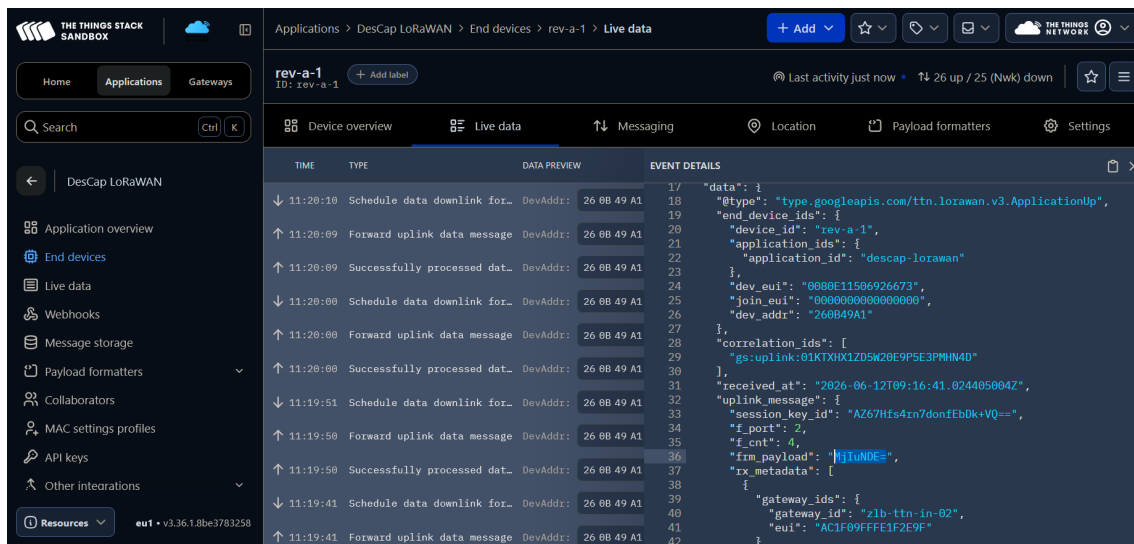


Abbildung 3.32: Screenshot von TTN-Konsole nach erfolgreichem Join-Request

Nachdem die Join-Anfrage über OTAA vom Netzwerkservers erfolgreich verarbeitet wurde, zeigt die TTN-Konsole für das Endgerät **rev-a-1** die korrekte DevEUI, JoinEUI sowie den Zeitstempel des Paketempfangs an (Abbildung 3.32). Diese Übereinstimmung mit den in der Firmware hinterlegten Werten bestätigt, dass das Aktivierungsverfahren wie spezifiziert abgelaufen ist.

Die in Zeile 36 sichtbare Payload wird im Base64-Format kodiert übertragen (**MjIuNDU=**) und entspricht im Klartext dem Temperaturwert „22.41“ (°C). Damit ist die End-zu-Ende-Datenintegrität von der Sensormessung über die Firmware-Kodierung bis zur Darstellung auf Serverseite nachgewiesen – eine Grundvoraussetzung für die in Kapitel ?? folgenden quantitativen Auswertungen.

Anhand der in der Konsole angezeigten Gateway-ID in Zeile 40 lässt sich das empfangende Gateway als das öffentliche TTN-Gateway der Zentral- und Landesbibliothek Berlin (ZLB) identifizieren, was damit übereinstimmt, dass die Join-Anfrage in unmittelbarer Nähe auf der Museumsinsel initiiert wurde.

3.5.2 Kartografische Evaluierung der TTN-Netzwerkabdeckung

- 4 große Gateways ausgewählt und auf der Karte markiert in himmelsblau (3 in Steglitz und eine in Mitte), - jedes GW mit ner Form (Kreis, Stern, Quadrat, Dreieck) - Messpunkte sind kleiner und farbig markiert je nach RSSI - Übereinstimmung mit der Karte von TTN-Mapper, was RSSI angeht
- Erwähnt maximale Reichweite: 2km in Steglitz und 6km aufm Tempelhofer Feld (vom Gateway ttn-outdoor-02 in Steglitz empfangen) - Von 973 Datenpunkten taucht 2km und 6km nur einmal auf, also wahrscheinlichkeit für weite Übertragungen gering - On-board L80R GPS funktioniert nicht weil zu lange Fix-Zeit - GPS-Logger App auf Handy als Alternative nötig - In Mitte viele

Messpunkte verloren gegangen wegen GPS Problem - Include graphics von ttn_log Positionen - Vllt ein paar Pics von den gateways im echten Leben - Erwähne maximale LoRaWAN-Reichweite in Städten - macht Sinn, weil ttn-outdoor-02 Antenne ist auf 90m Höhe - In Mitte bescheidene Reichweite (1km) wegen höherer Gebäudedichte - Vllt irgendwelche abrundenen Worte am Ende

Zur Visualisierung der Messergebnisse wurden vier ausgewählte Gateways auf der Karte hervorgehoben und in Himmelblau markiert: drei davon befinden sich in Steglitz, eines in Mitte. Um die Gateways optisch unterscheidbar zu machen, wurde jedem eine eigene geometrische Form zugeordnet (Kreis, Stern, Quadrat, Dreieck). Die eigentlichen Messpunkte sind als kleinere, nach empfangener Signalstärke (RSSI) farbig kodierte Marker dargestellt. Ein Vergleich mit der öffentlich einsehbaren Abdeckungskarte von TTN Mapper zeigt eine gute Übereinstimmung hinsichtlich der beobachteten RSSI-Verteilung im untersuchten Gebiet, was die Plausibilität der eigenen Messung zusätzlich unterstützt.

Die größte gemessene Reichweite betrug rund 6 km und wurde auf dem Tempelhofer Feld vom Gateway in Steglitz empfangen; im unmittelbaren Steglitzer Stadtgebiet selbst lag die maximale Reichweite bei etwa 2 km. Von insgesamt 973 erfassten Datenpunkten trat sowohl die 2-km- als auch die 6-km-Distanz jeweils nur ein einziges Mal auf, was darauf hindeutet, dass derart weite Übertragungen im urbanen Raum eher die Ausnahme als die Regel darstellen. Die vergleichsweise große Reichweite des Gateways `ttn-outdoor-02` lässt sich durch die Antennenhöhe von rund 90 m über Grund erklären, die eine deutlich bessere Sichtverbindung zu entfernten Endgeräten ermöglicht als bodennahe Installationen.

Im Gegensatz dazu fiel die maximale Reichweite in Mitte mit etwa 1 km deutlich geringer aus. Dies lässt sich ebenfalls durch die Antennenhöhe begründen, welche in diesem Fall lediglich 12 m beträgt. Dies lässt sich auf die dort wesentlich höhere Gebäudedichte zurückführen, die zu stärkerer Signaldämpfung und häufigerer Abschattung durch Sichtlinienhindernisse führt – ein Effekt, der sich mit der allgemein bekannten Einschränkung maximaler LoRaWAN-Reichweiten in dicht bebauten städtischen Umgebungen deckt.

Testplanung Funkabdeckung (vllt passt nicht in diesen Ergebnisse-Teil): - Routenplanung: + Zuerst gute Gateways aussuchen, um die die Routen konstruiert werden sollen + include graphics ttn mapper gateway+heatmap + 4 GWs festgelegt: 3 in Steglitz mit max 90m und 1 in Mitte mit 12m, 12m ist in der Nähe meiner Wohnung deswegen gewählt + Tabelle auf Basis von gateway_info.csv + danach Viewshed analysis mit QGIS, um die genauen Straßen um die Gateways zu planen + QGIS kurz vorstellen, was es macht aber nicht so werbungsartig schreiben + Viewshed Schritte: 1. DOM von Berlin runterladen, erklären was DOM ist (muss nicht an dieser Stelle sein, aber wo auch immer angemessen) 2. Ein paar Infos zu Berlin DOM: 297 2kmx2km Tiles

+ includegraphics rasterkarte 2. DOM Nachverarbeitung (wenn relevant für Arbeit), damit VRT entsteht -> VRT in QGIS 3. Gateways als Viewpoints festlegen 4. Viewshed Analysis + Für 6km radius analyse ungefähr 2 Stunden in QGIS + include graphics viewshed results + Schlussfolgerung: welche Straßen am besten + graphics of Route + Zusatz: Tempelhofer Feld ist weit weg von den GWs, aber erkläre trotzdem warum diese Route sinnvoll ist - Zeitliche Planung: + Nachts: nach 22h-1h möglichst wenig Störung, Route um zlb-Gateway in Mitte + Morgens: 10h- 12h GW in STeglitz - Durchführung: + Routen belaufen, während das PCB ständig ans TTN-netzwerk Daten schickt + GPS werden über L80R Modul auch geschickt, danach TTN-Message mit GPS-Daten matchen

3.5.3 Energieprofiling

3.6 Fazit und Ausblick

3.6.1 Ausblick

FUOTA: weil manche Endgeräte schwer zugänglich sind, und weil ... Solarzelle testen für autarken Betrieb

Entwurf

Entwurf

Anhang A

Angehängtes: Die Dateien des Pakets

Stylefile

Die Styledatei für diese Abschlussarbeit ist `bhtThesis.sty`, die in der Archivdatei vorliegt. Diese muss von $\text{\LaTeX}$ auffindbar sein, muss also in einem $\text{\LaTeX}$ bekannten Ordner liegen:

- Ubuntu-Linux: `$HOME/texmf/tex/latex/bhtThesis/bhtThesis.sty`
- MikTeX: `c:\localtexmf\tex\latex\bhtThesis\bhtThesis.sty`

Beispieldokument

Dieses Dokument befindet sich im Unterordner `tryout` des zip-files. Sie können diese Dateien in einen Ordner kopieren, in dem Sie schliesslich arbeiten werden. Die Dateien sind die folgenden

- `abstract_de.tex` Kurzfassung in deutscher Sprache
 - `abstract_en.tex` Kurzfassung in englischer Sprache
 - `anhang.tex` der Anhang
 - `bhtThesis.bib` beinhaltet die zu zitierenden Literaturstellen und wird von $\text{bib}\text{\TeX}$ ausgewertet
 - `main.pdf` ist die Ausgabendatei mit der Druckvorlage
 - `main.tex` beinhaltet das Hauptdokument
 - `makefile` realisiert das automatische mehrfache Übersetzen, hierfür muss `make` auf dem System installiert sein.
 - `myapaLike.bst` beinhaltet die Formatierung für das Literaturverzeichnis
-

- `personalMacros.tex` kann einzelne, persönliche Macros beinhalten, die das Schreiben erleichtern
- `titelseiten.tex` realisiert alle Seiten bis zum Beginn des ersten Abschnittes
- Ordner `pictures`
 - `BHT-Logo-Basis.eps`
 - `BHT-Logo-Basis.pdf`
- Ordner `kapitel1`
 - `ch1.tex` Quelltext des Kapitel 1
 - Ordner `pictures`
 - * `schaltbild.pdf`
- Ordner `kapitel2`
 - `ch2.tex` Quelltext des Kapitel 2
 - Ordner `pictures`
 - * `leer`

Entwurf

Literatur

- [1] J. R. Rumble, Hrsg., *CRC handbook of chemistry and physics*, eng, 102nd edition 2021-2022. Boca Raton London New York: CRC Press, 2021, ISBN: 978-0-367-41724-6 978-0-367-71260-0.
 - [2] L. Sciullo, A. Trotta und M. D. Felice, „Design and performance evaluation of a LoRa-based mobile emergency management system (LOCATE),“ en, *Ad Hoc Networks*, Jg. 96, S. 101993, Jan. 2020, ISSN: 15708705. DOI: 10.1016/j.adhoc.2019.101993. Adresse: <https://linkinghub.elsevier.com/retrieve/pii/S1570870518309004> (besucht am 25.03.2026).
 - [3] B. S. Chaudhari und M. Zennaro, Hrsg., *LPWAN technologies for IoT and M2M applications*, eng. London, England: Academic Press, 2020, ISBN: 978-0-12-818880-4 978-0-12-818881-1.
 - [4] „LoRa™ Modulation Basics“, Semtech, Application Note AN1200.02.
 - [5] „LoRaWAN® L2 1.0.4 Specification (TS001-1.0.4)“, LoRa Alliance, Technical Documentation TS001-1.0.4, Okt. 2020.
 - [6] „LoRaWAN® Backend Interfaces Technical Documentation (TS002-1.1.0)“, LoRa Alliance, Technical Documentation TS002-1.1.0, Okt. 2020. Adresse: https://loro-alliance.org/wp-content/uploads/2020/11/TS002-1.1.0-LoRaWAN_Backend_Interfaces.pdf.
 - [7] J. Tapparel, O. Afisiadis, P. Mayoraz, A. Balatsoukas-Stimming und A. Burg, „An Open-Source LoRa Physical Layer Prototype on GNU Radio“, in *2020 IEEE 21st International Workshop on Signal Processing Advances in Wireless Communications (SPAWC)*, Atlanta, GA, USA: IEEE, Mai 2020, S. 1–5, ISBN: 978-1-7281-5478-7. DOI: 10.1109/SPAWC48557.2020.9154273. Adresse: <https://ieeexplore.ieee.org/document/9154273/> (besucht am 30.03.2026).
-